Refer to the **insert** for the list of pseudocode functions and operators.

1 A user requires a program that implements complex data encryption to a recognised international standard. A programmer has started the design of the program.

(a) The programmer decides to use a library routine to provide part of the solution. One reason for this is that library routines may perform functions that the programmer is unable to program themselves.

State **three** other benefits of using library routines in the development of the program.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

(b) The programmer has identified a different part of the algorithm that would be appropriate to implement as a subroutine (a procedure or a function); no suitable library routine exists for this part.

(i) State **two** reasons why the programmer may decide to use a subroutine.

1 .....

.....

2 .....

.....

[2]

(ii) A function header in pseudocode is defined as:

```
FUNCTION Pass2(Count : INTEGER) RETURNS BOOLEAN
```

Complete the table by describing the terms used in the function header.

The first row has been completed.

Term	Description
`Pass2`	the name of the function
`Count`	
`BOOLEAN`	

[2]

(c) Variables in the program have example values:

Variable	Example value
DoB	23/6/2011
Multiplier	2.5
AddressLine[1]	"35 Lincoln Avenue"

Complete the table by evaluating each expression using the example values:

Expression	Evaluates to
LENGTH(NUM_TO_STR(Multiplier))	
MONTH(DoB) > 4	
15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	

[3]

2 **(a)** A program contains a global 1D array of type `STRING` containing 65 elements.

An existing text file is used to store the data in the array. Only non-blank elements (those that do **not** contain an empty string) are written to the text file.

An algorithm will:
- write the first element of the array as a new line in the text file
- continue until all elements have been written, each to a new line of the text file.

Stepwise refinement is applied to the algorithm.

Describe up to **six** steps for this algorithm that could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

Step 6 .....

.....

[6]

(b) Iteration is one programming construct.

Identify **one** other programming construct that will be required when the algorithm from part **(a)** is converted into pseudocode and explain how it is used.

Construct .....

.....

Use .....

.....

[2]

4 A program contains a global 1D array `Data` containing 20 elements of type `INTEGER`

A global string `NumString` represents a sequence of three-digit numbers, separated by commas. For example:

```
"101,456,219,754,328"
```

The string contains at least **four** three-digit numbers. The total number of three-digit numbers in the string is unknown.

A procedure `Store()` will:

- extract one three-digit number at a time from `NumString`
- convert each of the three-digit numbers extracted to an integer and assign this to the next array element, starting from index 1
- end when all three-digit numbers have been stored, or when the array is full.

Complete the pseudocode for `Store()`

All local variables used must be declared.

```
PROCEDURE Store()
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[6]

5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
Num[2] ← 2
Num[3] ← 5
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
   DECLARE CaseVar, Index, Count : INTEGER

   Index ← Start
   Count ← 0

   WHILE Count <= 20
      CaseVar ← Num[Index]
      CASE OF CaseVar
         1 : Num[Index] ← Num[Index] + Index //clause one
             Index ← Index + 1
             Count ← Count + 1

         2 : Num[Index] ← Num[Index] + Index //clause two
             Index ← Index + 2
             Count ← Count + 2

         3 : Num[Index] ← Num[Index] * 2     //clause three
             Index ← Index + 3
             Count ← Count - 1

         4 : Count ← Count + 4               //clause four
         OTHERWISE : Count ← 20
      ENDCASE

      Index ← (Index MOD 4) + 1

   ENDWHILE

ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called in the statement:
`CALL Process(1)`

Index	CaseVar	Count	Num[1]	Num[2]	Num[3]	Num[4]

[5]

(b) As a reminder, the `CASE` structure in the pseudocode is:

```
CASE OF CaseVar
   1 : Num[Index] ← Num[Index] + Index //clause one
       Index ← Index + 1
       Count ← Count + 1

   2 : Num[Index] ← Num[Index] + Index //clause two
       Index ← Index + 2
       Count ← Count + 2

   3 : Num[Index] ← Num[Index] * 2     //clause three
       Index ← Index + 3
       Count ← Count - 1

   4 : Count ← Count + 4               //clause four
   OTHERWISE : Count ← 20
ENDCASE
```

The `CASE` structure could be optimised by combining two existing clauses.

(i) Identify the `CaseVar` values for these **two** existing clauses.

..... [1]

(ii) Write pseudocode for a single clause to replace the **two** clauses identified in **(b) (i)**.

.....

.....

..... [2]

6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	4 + 1 + 6 + 2 = 13
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

(a) A function `Generate()` is written to take an original number as a parameter and to return the check digit.

Outline a test plan that could be used to fully test function `Generate()`

Assume that the parameter is valid.

.....

.....

.....

..... [2]

(b) A function `CheckNumber()` will take an integer value and return the Boolean value `TRUE` if the check digit is correct, otherwise return `FALSE`

Write pseudocode for the function `CheckNumber()`

Assume that the parameter is valid.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book The first three characters of a `StudentID` represent a tutor ID. Each student has one tutor.
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 8000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
`LoanStatus()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • outputs a message saying whether a given loan has been returned or not • outputs a warning message if a record of the given loan is **not** found

(a) Write efficient pseudocode for the module `LoanStatus()`

Assume that each combination of `StudentID` and `BookID` can occur only once.

[8]

(b) A second module is defined:

Module	Description
`LoansPerTutor()`	• called with a parameter of type `STRING` representing a tutor ID (as a reminder, the first three characters of a `StudentID` represent a tutor ID) • returns an integer value representing the number of books currently on loan to students who have the given tutor

Reminder: unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

Write pseudocode for the module `LoansPerTutor()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....[7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** The table contains pseudocode examples.

Each example may contain statements that relate to one or more of:

- selection
- iteration (repetition)
- subroutines (procedures or functions).

Complete the table by placing **one or more** ticks (‘✓’) in each row.

Pseudocode example	Selection	Iteration	Subroutine

`IF Status = FALSE THEN`
`  FOR Count ← 1 TO 20`
`  CALL Reset(Count)`
`  NEXT Count`
`ENDIF`			

`OTHERWISE : Status ← TRUE`			
`WHILE AllDone() = TRUE`			
`30 : NextChar ← 'X'`			

[4]

(b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
`Result`	`5.42`	
`MonthLetter`	`"JFMAMJJASOND"`	
`Birthday`	`15/11/2009`	

[3]

(c) Evaluate each expression in the table by using the data values shown in **(b)**.

Write ‘ERROR’ if the expression contains an error.

Expression	Evaluates to
`INT(Result) + 1 > 6`	
`NUM_TO_STR(LENGTH(MonthLetter))`	
`NUM_TO_STR(Result + "3.2")`	
`MID(MonthLetter, MONTH(Birthday) – 2, 1)`	

[4]

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	`TRUE`	valid total score (no correct answers)
1	`FALSE`	invalid total score
2	`FALSE`	invalid total score
3	`TRUE`	valid total score (one 3-point question correct)
4	`FALSE`	invalid total score
5	`TRUE`	valid total score (one 5-point question correct)
6	`TRUE`	valid total score (two 3-point questions correct)
7	`FALSE`	invalid total score
8	`TRUE`	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(a) Write pseudocode to declare `CheckTotal` and to set all elements of the array to `FALSE`

All variables used must be declared.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(b) The pseudocode represents an algorithm to set the appropriate elements of the array to `TRUE`

Complete the pseudocode:

```
DECLARE EasyQ, HardQ : INTEGER

FOR EasyQ ← ..... TO 15 STEP .....

   FOR HardQ ← 0 TO ..... STEP .....

      CheckTotal[.....] ← TRUE

   NEXT HardQ

NEXT EasyQ
```

[5]

(c) The array elements have been assigned the required values.

A module `ValidateScore()` will take an integer value representing a total score as a parameter. It will return `TRUE` if the total score is valid, or `FALSE` if it is **not** valid.

Describe the algorithm for `ValidateScore()` using **four** steps.

Do **not** use pseudocode in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

[4]

6 A string function `Compare()` will compare two strings.

The function will:

- take **four** parameters:
  - two strings, `String1` and `String2`
  - a character, `Position`, to indicate whether `String1` will be compared with the start (`'s'`), or the end (`'e'`) of `String2`
  - a Boolean, `CaseMatters`, to indicate whether an upper case character and lower case character (for example, `'A'` and `'a'`) are regarded as different (`TRUE`), or as the same (`FALSE`)
- return `FALSE` if `String2` has fewer characters than `String1`
- return `TRUE` if the comparison is true, otherwise return `FALSE`

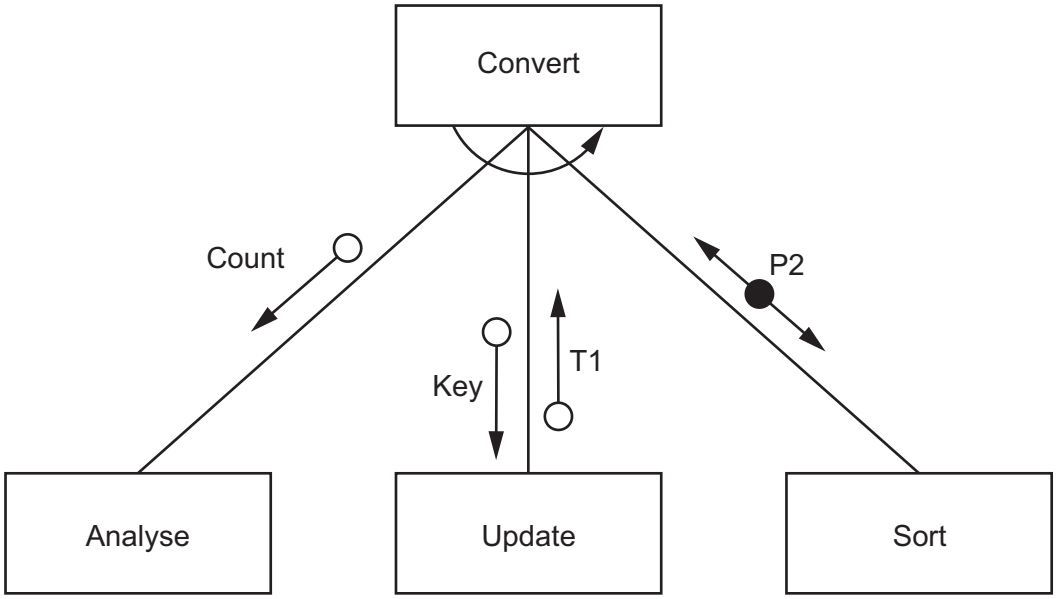
For example:

Parameter				Return value
String1	**String2**	**CaseMatters**	**Position**	
"Cat"	"Catalogue"	TRUE	's'	TRUE
"CAT"	"Catalogue"	TRUE	's'	FALSE
"CAT"	"Catalogue"	FALSE	's'	TRUE
"Cat"	"Catalogue"	TRUE	'e'	FALSE
"GUE"	"Catalogue"	FALSE	'e'	TRUE
"Catalogue"	"Cat"	TRUE	's'	FALSE

Write pseudocode for the function `Compare()`

[7]

7 The structure chart shows part of a program design:



(a) Explain the meaning of the curved arrow symbol in this structure chart.

.....

.....

.....

..... [2]

(b) The program designer has noted that:

- Count is the number of elements in an array
- T1 is a value representing the number of elements in an array that have been modified
- Key is of type `STRING`

Write the pseudocode module headers for analyse, update and sort.

Analyse:

.....

.....

Update:

.....

.....

Sort:

.....

.....

[4]

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
`CountLoans()`	• called with a parameter of type `STRING` representing a `StudentID` • counts the number of books currently on loan to the specified student • counts the number of books that the student has already returned • output both counts together with a suitable message

(a) Write pseudocode for module `CountLoans()`

.....

.....

.....

(b) As a reminder, a global array `Loan` stores 7000 loan records with data items for each loan record:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

A new module is defined:

Module	Description
`NewLoan()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • searches the array for an unused loan record • if found, updates the loan record and returns `TRUE`, otherwise returns `FALSE`

Write efficient pseudocode for `NewLoan()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed to meet a particular customer requirement.

(a) The program contains these variables:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER
MyDOB	DATE

Complete the table by filling in the gaps using functions and/or operators from the **insert**.

Each expression must be valid.

Expression
MyString ← 'X' ..... MyString
MyChar ← ..... ("ABCD", ... ,.....)
MyString ← ..... (..... (MyDOB))
MyInt ← ..... (..... (MyString) / 2)

[4]

(b) State **three** benefits of using an array of records to store the data for all rentals.

1 .....

.....

2 .....

.....

3 .....

.....

4 A program contains a global 1D array `Number` consisting of 20 elements of type `REAL`

A procedure `Store()` will input a sequence of up to 20 real values, one value at a time. These values will be assigned to elements of the array using four steps:

Step 1: store the first value in the sequence in the first element of the array
Step 2: check each subsequent value input. If this value is larger than the previous value input, then assign the value to the next array element, otherwise go to **step 4**
Step 3: repeat from **step 2** unless the array is full
Step 4: output the count of the number of values stored in the array together with a suitable message.

(a) Complete the pseudocode for `Store()`

All variables used in the algorithm must be declared.

```
PROCEDURE Store()
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDPROCEDURE
```

[6]

(b) The requirements of the program change:

- the number of values in the sequence is unknown, but may be higher than 20
- the values will need to be accessed by another program as data.

The data will be stored in a text file instead of an array.

(i) Give **two** benefits of using a text file instead of an array.

1 .....

.....

2 .....

.....

[2]

(ii) State any change that will need to be made before each value is written to the file.

.....

..... [1]

5 A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return –1 if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
   DECLARE Index, FoundAt : INTEGER
   FoundAt ← -1
   FOR Index ← 1 TO 10000
      IF MyData[Index] = SearchString THEN // outer conditional clause
         IF FoundAt = -1 THEN              // inner conditional clause
            FoundAt ← Index
         ENDIF
      ENDIF
   NEXT Index
   RETURN FoundAt
ENDFUNCTION
```

(a) (i) Comment on why the programmer chose –1 as the initial value of `FoundAt`

.....

..... [1]

(ii) Explain the purpose of the **outer** conditional clause.

.....

.....

..... [1]

(iii) The **inner** conditional clause ensures that only the index of the **first** matching element, if any, is returned.

Explain how this clause works.

.....

.....

..... [1]

(b) The pseudocode does **not** use the most appropriate loop construct.

(i) Explain why this is **not** the most appropriate loop construct.

.....

.....

..... [1]

(ii) Suggest and justify a more appropriate loop construct that could be used.

Construct .....

Justification .....

.....

[2]

6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	4 + 1 + 6 + 2 = 13
remainder when the sum is divided by 10 using integer division	3
new number	41623

The method described can be used to detect single-digit errors. For example, if the new number is incorrectly input as 41633, then the check digit does not match and the input will be rejected.

(a) An incorrect attempt was made to enter 41623. The **first two** digits were entered incorrectly; the **last three** digits were entered correctly.

When this number was tested, the check digit was found to be correct and the input was accepted.

Identify an example of the incorrect attempt to enter 41623 and explain why this number would **not** be rejected.

Number .....

Explanation .....

.....

[2]

(b) A module `Generate()` will take an integer value representing an original number and return an integer value representing a new number which includes the check digit.

The original number is always at least three digits in length.

Write pseudocode for the module `Generate()`

Assume that the parameter is valid.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

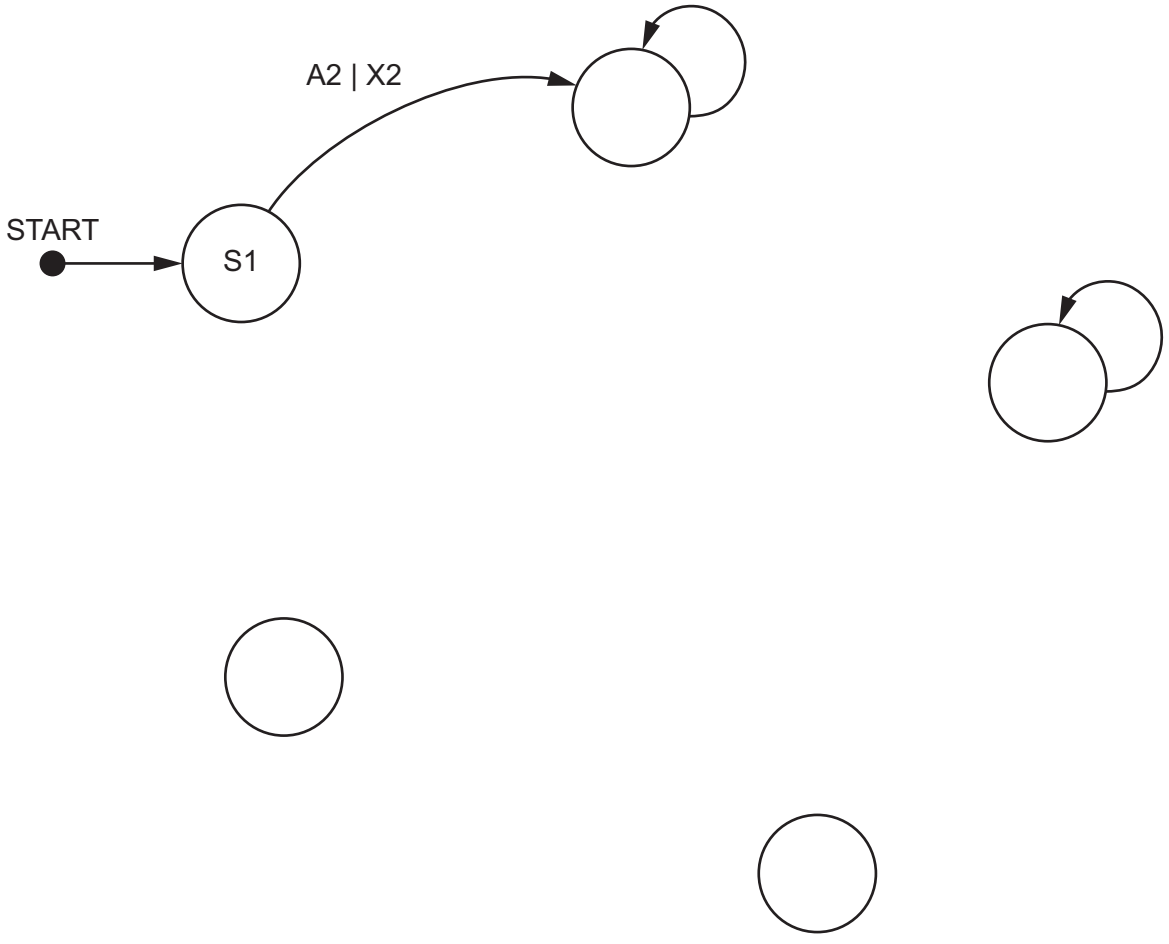
7 There are several different ways to express an algorithm during the design of a program.

(a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A2	X2	S3
S1	A1	X1	S2
S2	A4	X4	S5
S3	A1		S3
S3	A3	X3	S2
S3	A2	X4	S4
S4	A1	X1	S4
S4	A3		S2
S4	A4	X4	S5

Complete the state-transition diagram to represent the information given in the table.



[5]

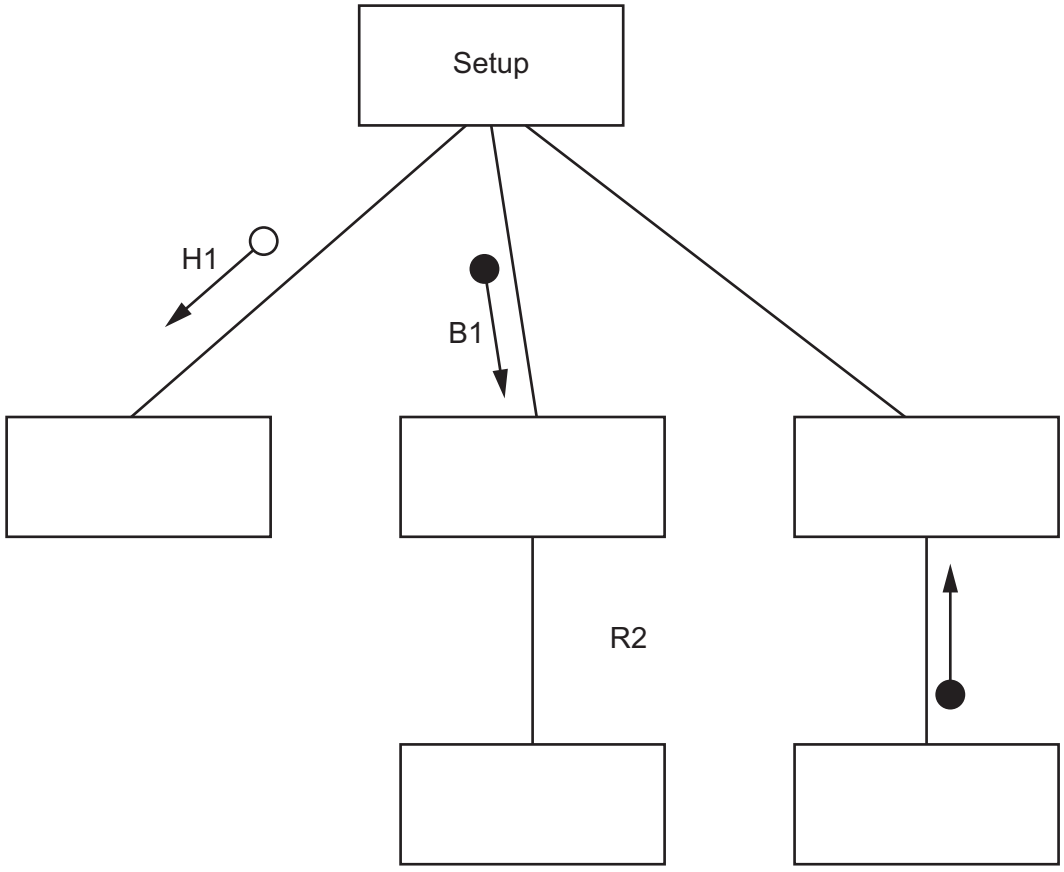
(b) A structure chart is used to document a different part of the program.

This part of the program contains six modules:

Pseudocode module header
`PROCEDURE Setup()`
`PROCEDURE Restart(H1 : STRING, C1 : INTEGER)`
`FUNCTION Modify(B1 : BOOLEAN) RETURNS INTEGER`
`PROCEDURE Final(T1 : INTEGER)`
`PROCEDURE Update(BYREF R2 : STRING)`
`FUNCTION Confirm() RETURNS BOOLEAN`

Module Setup will repeatedly call **three** of the modules.

Complete the structure chart to document the information given in the table.



[5]

8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
`OKToBorrow()`	• called with a parameter of type `STRING` representing a `StudentID` • search the array for loan records for the specified student • output a suitable message to say whether the student may, or may not, borrow another book

(a) Write efficient pseudocode for the module `OKToBorrow()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A second module is defined:

Module	Description
`ReturnBook()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • searches the array for the relevant loan record • when found, sets `OnLoan` to `FALSE` and returns `TRUE` • if a loan record is **not** found, or the book has already been returned, then returns `FALSE`

Write efficient pseudocode for the module `ReturnBook()`

Assume that each student is only allowed to borrow each book only once. That means that there will be no more than **one** loan record for a given combination of student and book.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed for the management of a sports centre.

(a) The programmer developing the software decides to use modules (procedures or functions) and local variables. They also decide not to use global variables.

(i) State **two** reasons why the programmer decides to use modules.

1 .....

.....

2 .....

.....

[2]

(ii) State **one** difference between local and global variables.

.....

..... [1]

(iii) State **two** benefits of using local variables.

1 .....

.....

2 .....

.....

[2]

(b) The pseudocode design contains a number of expressions.

Complete each pseudocode expression so that it evaluates to the value shown.

Refer to the **insert** for the list of pseudocode functions and operators.

Expression	Evaluates to
.....(68)	'D'
.....(04/02/2025)	2
.....TRUE	FALSE
.....(..... ("Court1.4Upper" ,.....,.....))	1.4

[5]

3 A programmer has been asked to create a module `RollDice()` to simulate multiple rolls of a dice. This module will be used as part of a program for a game.

The module will:

step 1 – take a positive integer parameter representing the number of times the dice will be rolled

step 2 – simulate **one** roll of the dice by generating a random integer between 1 and 6 inclusive

step 3 – output the integer generated

step 4 – repeat, as required from **step 2**

step 5 – calculate the average value of the random integers generated

step 6 – return the average value.

Write pseudocode for the module `RollDice()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

5 A module `Parity()` takes a string as a parameter. The parameter has the identifier `BitString` and it represents a binary value.

The module will concatenate a single character to the end of `BitString` by applying one of the two rules:

- `'0'` if `Bitstring` contains an even number of 1s
- `'1'` if `Bitstring` contains an odd number of 1s.

The modified value of `BitString` is then returned.

For example:

Parameter	String returned	Explanation
`"0010010110"`	`"00100101100"`	there are an even number of 1s in the string passed to `Parity()` so a `'0'` is concatenated to the end of `BitString`
`"101010"`	`"101010**1**"`	there are an odd number of 1s in the string passed to `Parity()` so a `'1'` is concatenated to the end of `BitString`

Write pseudocode for the module `Parity()`

Assume the parameter `BitString` can only contain the characters `'0'` and `'1'`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

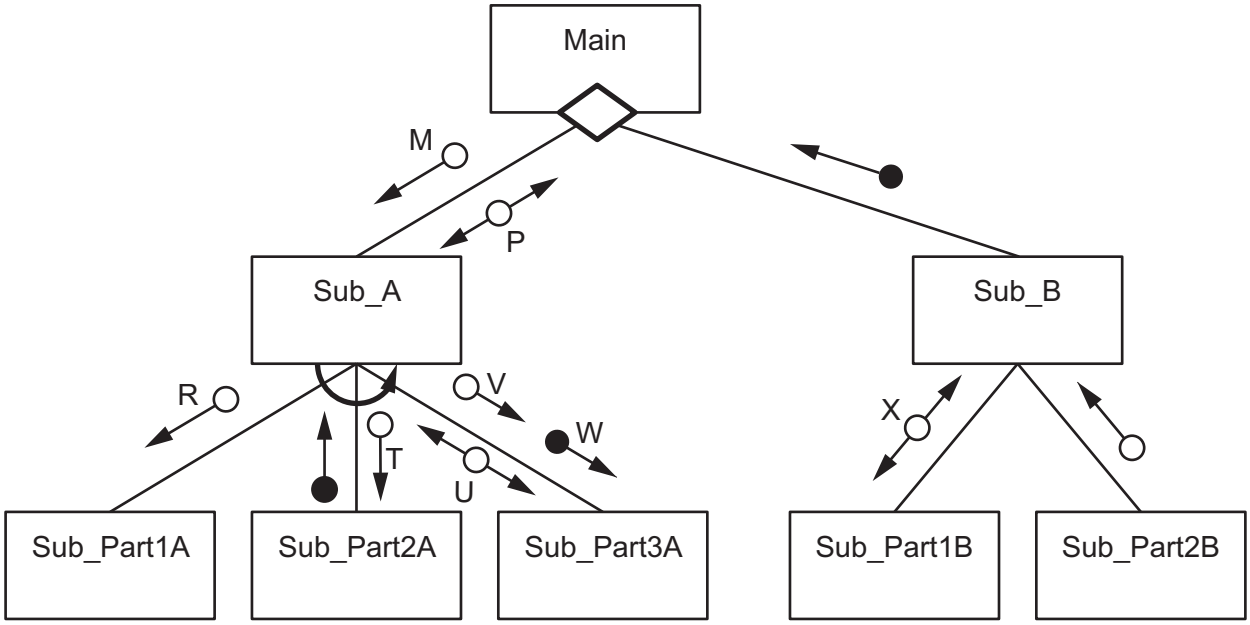
.....

.....

.....

..... [8]

6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(a) Explain **one** difference between functions and procedures.

.....

..... [1]

(b) Write the pseudocode to define the module headers:

Sub_Part1A

.....

.....

Sub_Part2A

.....

.....

Sub_Part3A

.....

.....

[5]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
`UpdateVisit()`	• called with two parameters: 1. loyalty points of type `INTEGER` 2. last visit date of type `DATE` • change the loyalty points: ○ increase by **four** if the last visit date and the current date are in the same month ○ otherwise increase by **one** • change the last visit date to the current date • the changed values must be available to the code that follows the call to `UpdateVisit()`

(a) (i) Write pseudocode for module `UpdateVisit()`

Assume the customer has made at least **one** previous visit.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.... [5]

(ii) The programmer decides to amend the `UpdateVisit()` module so that loyalty points are further increased if the customer visits the coffee shop the same day of the week as their last visit.

Write the pseudocode for this condition.

.....

..... [1]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: DDMMYYYY.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

```
"100123,132,05032025"
```

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
`FindCustomer()` (is already written)	• called with a parameter of type `STRING` that represents a customer ID • returns the line of data read from the text file `Loyalty.txt` containing the data items for that customer
`MondayCheck()`	• called with a parameter of type `STRING` that represents the customer ID of a customer • calls `FindCustomer()` • extracts the loyalty points from the string returned by `FindCustomer()` • increases the loyalty points for the current customer by 10 if the day visited is a Monday • returns an integer value representing the loyalty points

(i) Write pseudocode for module `MondayCheck()`

The module `FindCustomer()` must be used and assume it returns a valid string for the current customer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

(ii) A date is stored in `LastVisitDateString` in the format:

DDMMYYYY

DD is a 2-digit string
MM is a 2-digit string
YYYY is a 4-digit string.

For example, the date 03/09/2024 is stored as `"03092024"`

An algorithm is needed to convert the string stored in `LastVisitDateString` to data type `DATE` which is then stored in the variable `LastVisitDate`.

Complete the pseudocode for this algorithm:

Assume that `LastVisitDateString` has been declared and contains valid data.

```
DECLARE DayInt, MonthInt, YearInt : INTEGER

DECLARE LastVisitDate : DATE
```

.....

.....

.....

.....

.....

.....

.....

.....

[4]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed to control the production line in a factory.

(a) A number of different program life cycles are available for the development of a program.

(i) Explain the need for different program development life cycles.

.....

..... [1]

(c) Pseudocode has been used to design modules for the program to control the production line.

The table shows **four** valid pseudocode expressions.

Complete the table by giving the data type of the evaluated expression.

Expression	Data type
`RIGHT(MachineCode, 4)`	
`Speed * 2.5`	
`NOT Status`	
`IS_NUM(Check)`	

[4]

3 An algorithm is designed to generate and output two unique random integers. Each integer value is between –10 and 10 inclusive.

If both integers output are negative, a third random integer between 30 and 35 inclusive will be generated and output.

Write pseudocode for this algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

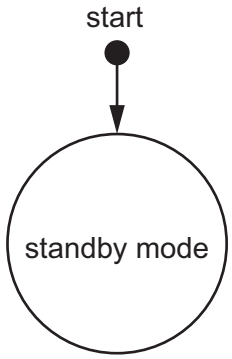
5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

(a) Complete the state-transition diagram for the automated digital camera system.



[4]

(b) At the end of each sequence of pictures, the time is saved as a string in the format `<HH><MM><SS>` where:

- `HH` represents the hours using two digits
- `MM` represents the minutes using two digits
- `SS` represents the seconds using two digits.

For example:

- `"081230"` is stored to represent the time 8:12:30, in the morning
- `"152235"` is stored to represent the time 15:22:35, in the afternoon.

Each string is stored on a new line in the text file `TimeTaken.txt`

An algorithm is required to process the content of the text file `TimeTaken.txt` once it has been transferred to the computer.

For each hour when pictures are taken, output a suitable message showing the hour and the total number of sequences of pictures taken within that hour.

Example outputs:

```
Hour : 15  Total : 32
Hour : 18  Total : 1
```

Write pseudocode for this algorithm.

Assume the text file `TimeTaken.txt` contains at least **one** line.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

6 Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

	Data
1	1018
2	1007
3	1010
4	1056
5	1092
6	1062
7	1034
8	0
9	0
10	0

	Pointer
1	7
2	3
3	1
4	6
5	-1
6	5
7	4
8	9
9	10
10	-1

Study the pseudocode:

```
PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
   DECLARE New, Current, Last : INTEGER
   CONSTANT NullPointer = -1
   New ← Unused
   Last ← NullPointer
   Current ← Start
   WHILE Current <> NullPointer AND Data[Current] < Value
      Last ← Current
      Current ← Pointer[Current]
   ENDWHILE
   Pointer[New] ← Pointer[Last]
   Pointer[Last] ← New
   Data[New] ← Value
ENDPROCEDURE
```

(a) (i) Complete the trace table below by dry running the procedure `Place()` when it is called by the statement:

```
CALL Place(1043, 2, 8)
```

Value	Start	Unused	New	Last	Current

[4]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

Each customer has a unique customer ID starting at `10001` with this value increasing by one each time a new customer joins the loyalty scheme.

For example, the third customer who joins the loyalty scheme is given the customer ID `10003`

The loyalty scheme is limited to 1000 customers.

A customer is awarded a loyalty point every time they buy a coffee.

The programmer has decided to use a global 2D array `Loyalty` of type `INTEGER`. The array `Loyalty` is made up of 1000 rows and 2 columns. Each row relates to one customer; column 1 contains the unique customer ID and column 2 contains the number of customer loyalty points.

Rows in the array `Loyalty` that are not currently being used have the value of Column 1 set to `99999`

The array is sorted in ascending order by customer ID.

The programmer has defined a program module:

Module	Description
`FindCustomer()`	• called with parameter of type `INTEGER` representing a customer ID • searches the `Loyalty` array for this customer ID • the search will stop as soon as the customer ID is found • the search should efficiently deal with the situation when the customer ID is **not** stored in the `Loyalty` array • if the customer ID is found, return an integer value representing the loyalty points, otherwise return `-1`

(a) Write efficient pseudocode for module `FindCustomer()`

[8]

(b) A customer can claim a free coffee for every 11 loyalty points.

The programmer has defined a second program module:

Module	Description
`PointsReport()`	• output the customer ID for each customer who has 11 or more loyalty points • output the average loyalty points for all customers in the `Loyalty` array along with an appropriate message

Write efficient pseudocode for module `PointsReport()`

Assume the array contains the data for at least **one** customer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed to help the manager of a shop control the stock.

(a) An identifier table has been used during the design stage.

Complete the identifier table:

Example value	Explanation	Variable name	Data type
`"Fruit"`	a category of stock that is sold in the shop		
`20/02/2025`	when an item was sold		
`12.67`	the cost of an item		
`TRUE`	to indicate if an item is in stock		

[4]

(b) A module `Sales()` is part of the stock control program.

The table contains pseudocode extracts from the module `Sales()`

Each extract may include all or part of:

- assignment
- selection
- iteration (repetition).

Complete the table by placing one or more ticks (✓) in each row:

Pseudocode extract	Assignment	Selection	Iteration
`Result ← CalculateTotal()`			
`WHILE IsClosed`			

`REPEAT`
`  INPUT Value`
`UNTIL Sales[4] > Value`			

`IF Sales[Current] <= 150 THEN`
`  Discount ← TRUE`
`ENDIF`			

`CASE OF Option`			

[5]

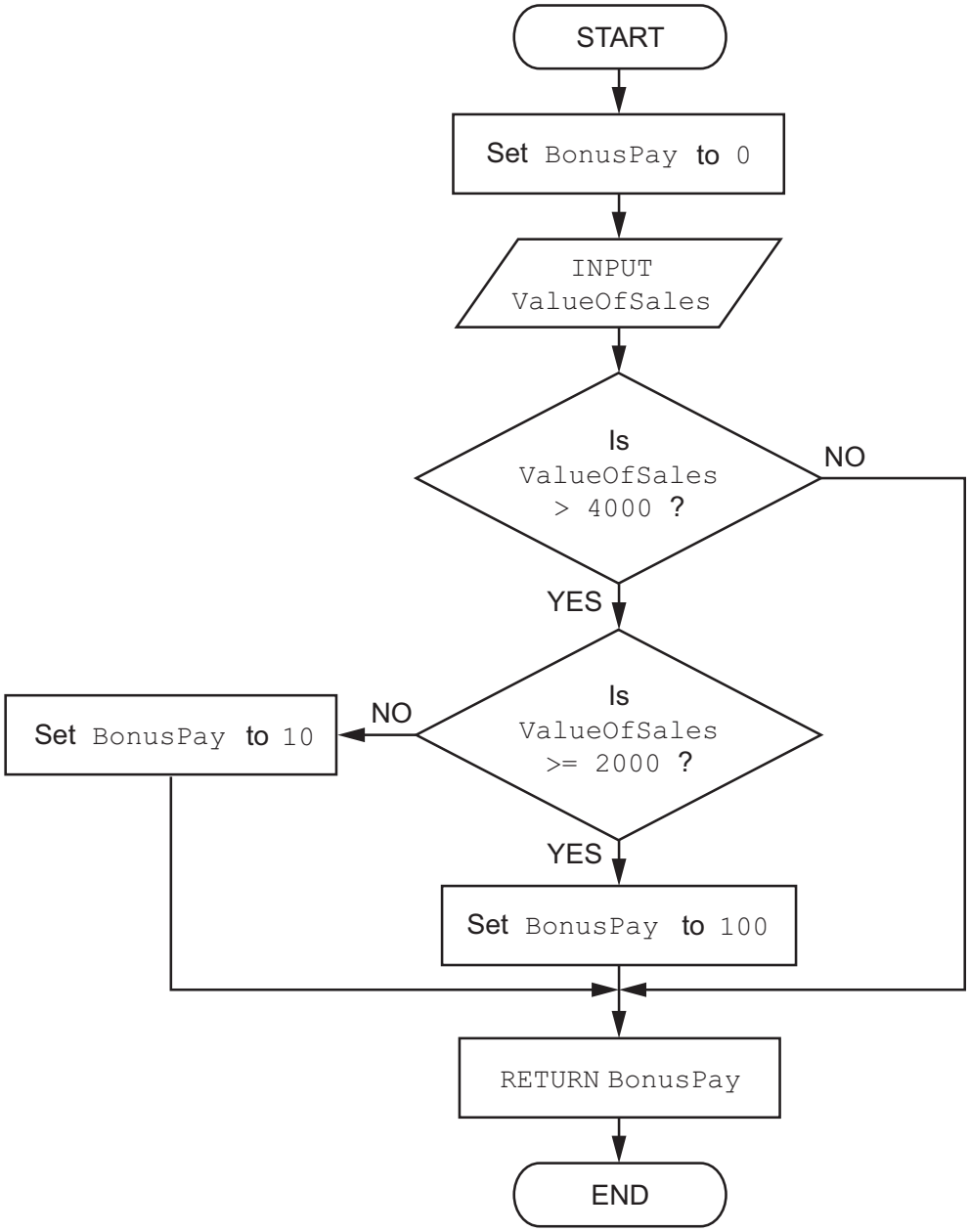
2 A program is being developed to calculate the pay of employees working for a company.

A function `CalculateBonus()` calculates bonus pay based on the value of sales.

Bonus pay is calculated as shown in the table.

Value of sales (in dollars)	Bonus pay (in dollars)
below 2000	0
between 2000 and 4000 inclusive	10
above 4000	100

A flowchart for the function `CalculateBonus()` has been designed.



(a) The flowchart contains logic errors.

One logic error is that `BonusPay` will **not** be set to `10` although the `ValueOfSales` input is between `2000` and `4000` inclusive.

(i) Explain why this error occurs.

.....

..... [1]

(b) There are different ways to reduce the risk of errors when developing the new program, such as the use of constants.

(i) State a value that could be replaced by a constant in the function `CalculateBonus()`

..... [1]

(ii) Explain how the use of constants helps to reduce the risk of programming errors.

.....

.....

.....

..... [2]

(iii) One other way that can reduce the risk of errors when writing the program is the use of library routines.

Explain how library routines can reduce the risk of programming errors.

.....

..... [1]

3 A student has been asked to create a simple guessing game program. This program will generate a random integer value between 1 and 100. It will then repeatedly prompt the user to input an integer value until they input the randomly generated value.

The student has written a structured English description:

step 1 – randomly generate an integer value between 1 and 100 inclusive
step 2 – prompt the user to input an integer value
step 3 – output an appropriate message if the value input was too high; then repeat from **step 2**
step 4 – output an appropriate message if the value input was too low; then repeat from **step 2**
step 5 – output an appropriate message if the value input was the same value that was randomly generated; then end the program.

Write a pseudocode algorithm from this structured English description.

Assume no input validation is needed.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

4 Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I <= 4                          //Outer loop
   Key ← Chars[I]
   J ← I - 1
   WHILE J >= 0 AND Chars[J] > Key     //Inner loop
      Chars[J + 1] ← Chars[J]
      J ← J - 1
   ENDWHILE
   Chars[J + 1] ← Key
   I ← I + 1
ENDWHILE
```

(a) The outer loop structure used in the algorithm is **not** the most appropriate one to use.

State the type of loop structure that would be the most appropriate to use and justify why it is the most appropriate.

Loop structure .....

Justification .....

.....

[2]

(b) Complete the trace table by dry running the algorithm.

The first row has been completed.

I	Key	J	Chars[J]	Chars			
				[1]	[2]	[3]	[4]
2				'D'	'T'	'H'	'R'

[6]

6 A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

(a) A test plan is needed to fully test the module.

Complete the table. The first line has been completed for you.

Assume the sensors generate only integer values.

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted

[4]

(b) Each sensor used in the system has a unique sensor ID number in the range 1 to 50.

The program that is used to monitor the speed of each vehicle reads each sensor value every second and stores this value along with the sensor's unique ID into a global 2D array `Reading`

The global array `Reading` has been declared as follows:

```
DECLARE Reading : ARRAY[1:2000, 1:2] OF INTEGER
```

The array contains 4000 elements organised as 2000 rows and 2 columns.

Column 1 contains the sensor value, and column 2 contains the sensor ID.

When 2000 sensor readings have been taken, the array is full and the system stops taking any more sensor readings until the array has been processed.

A procedure `Sort()` is needed to sort the array into ascending order of sensor value using an efficient bubble sort algorithm.

Write **efficient** pseudocode for the procedure `Sort()`

[8]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
  - the cup of coffee they have just ordered is given to them free of charge
  - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
  - they get one coffee free of charge for every 11 points
  - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
`CustomerOrder()`	• called with two integer parameters: ◦ the number of coffees ordered ◦ the current number of points • output a suitable message giving the number of free coffees • return a value for the new points total.

(a) Write pseudocode for module `CustomerOrder()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

```
<CustomerID>,<Points>
```

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at `"100001"` with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

```
"100205,0"
```

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
`AddNewCustomers()`	• called with an integer parameter representing the number of new customers to be added to the loyalty scheme • adds a new line, containing the required information, to the text file `Loyalty.txt` for each customer added to the loyalty scheme • outputs each new customer ID added to the loyalty scheme

(i) Write pseudocode for module `AddNewCustomers()`

Assume that there is at least **one** customer already in the loyalty scheme.

.....

.....

.....

.....

.....

.....

.....

....

.....

.....

.....

1 **(a)** The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- subroutine (procedure or function).

Complete the table by placing **one or more** ticks ('✓') in each row.

Pseudocode example	Selection	Iteration	Subroutine

`FOR Index ← 1 TO 3`
`  IF Safe[Index] = TRUE THEN`
`  Flap[Index] ← 0`
`  ENDIF`
`NEXT Index`			

`CASE OF Compound(3)`			
`REPEAT UNTIL AllDone() = TRUE`			
`WHILE Result[3] <> FALSE`			

[4]

(b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
`Available`	`TRUE`	
`Received`	`"18/04/2021"`	
`Index`	`100`	

[3]

(c) Evaluate each expression in the table by using the data values shown in part **(b)**.

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
`Available AND NOT(Index > 100)`	
`Index MOD 30`	
`NUM_TO_STR(Index + "33")`	

[3]

2 An algorithm will:

1. prompt and input a sequence of 100 integer values, one at a time
2. sum the positive integers
3. output the result of the sum.

(a) Write pseudocode for the algorithm.

Assume the value zero is neither positive nor negative.

You must declare all variables used in the algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

BLANK PAGE

4 An examination paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary.

A program is being written to process examination marks.

The five grade boundaries are stored in a global 1D array `GB` of type `INTEGER`, for example:

Index	Value	Comment
1	65	The minimum mark for an A grade.
2	57	The minimum mark for a B grade.
3	43	The minimum mark for a C grade.
4	35	The minimum mark for a D grade.
5	27	The minimum mark for an E grade.

Any paper that achieves a mark within 2 marks of a grade boundary must be checked. Using the given table, a paper with 45 marks would need to be checked.

(a) The pseudocode algorithm to determine whether a paper should be checked is as shown. The mark for the paper is stored in variable `Mark`. Global variables `Mark`, `Index`, `Upper` and `Lower` are declared as integers.

Complete the pseudocode.

```
FOR Index ← 1 TO .....
   Lower ← GB[Index] - 2
   Upper ← .....
   IF Mark ..... AND Mark ..... THEN
      OUTPUT "Check this paper"
   ENDIF
NEXT Index
```

[4]

(b) An alternative algorithm to determine if a paper needs to be checked uses a global 1D array `Check`, containing 76 elements of type `BOOLEAN`. The indices of the array are from 0 to 75 (inclusive), corresponding to the range of possible marks.

An element value in `Check` is `TRUE` if the index is within 2 marks of a grade boundary. For example, in the case where the C grade boundary is 43 the corresponding part of the `Check` array would be as follows:

Index	Value	
40	FALSE	
41	TRUE	
42	TRUE	
43	TRUE	← The grade boundary for a C grade
44	TRUE	
45	TRUE	
46	FALSE	

(i) The mark for a given paper is stored in variable `Mark`.

Describe how an algorithm would use the `Check` array to determine whether this paper should be checked.

.....

.....

.....

..... [2]

(ii) A procedure `GBInitialise()` will initialise the `Check` array using values from the `GB` array.

Note it can be assumed that the maximum grade boundary value for A is 70 and the minimum value for E is 15.

Write pseudocode for the procedure.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

6 In some countries, on the third Sunday in March, daylight saving time begins when clocks move forward by one hour.

A module `AdjustClock()` will take an integer parameter representing a year. The module will return an integer value representing the number of the day in March on which the clocks move forward.

For example, the following line of pseudocode would assign `DayNumber` the value 20:

```
DayNumber ← AdjustClock(2022)
```

Write pseudocode for the function `AdjustClock()`.

Date functions from the **insert** should be used in your solution.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

BLANK PAGE

8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Skill level	14, 23, 76	An integer in the range 0 to 100, inclusive.

The programmer has defined a record type to define each character. The record type definition is shown in pseudocode as follows:

```
TYPE CharacterType
   DECLARE Player : INTEGER
   DECLARE Role : STRING
   DECLARE Name : STRING
   DECLARE SkillLevel : INTEGER
ENDTYPE
```

The `Player` field indicates the player to which the character is assigned (1 to 6). This field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data, as follows:

```
DECLARE Character : ARRAY[1:45] OF CharacterType
```

At the start of the game all record fields are initialised, and all `Player` fields are set to 0

The programmer has defined a program module as follows:

Module	Description

`Count()`	• called with two parameters:
	  ○ an integer representing a player
	  ○ a string representing a character role
	• searches the `Character` array for characters with the given role that are assigned to the given player
	• counts the number of assigned characters and sums their total skill level
	• outputs the result of the search if characters with the given role are found, for example:
	`"Player 3 has 4 characters with the role of Teacher and the total skill level is 65"`
	• if no characters with the given role are found, outputs:
	`"No characters with that role are assigned to this player"`

(a) Complete the pseudocode for module `Count()`.

```
PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[7]

(b) The `Character` array data has been saved in the text file `SaveFile.txt`
Each line of the file contains one element of the array (one record).

New modules are defined:

Module	Description
`Extract()` (already written)	• called with two parameters: ○ a string representing a complete line from the text file ○ an integer representing a field number (see structure below) • returns a string representing the required field
`Restore()`	• opens the text file `SaveFile.txt` • reads lines from the file and assigns values to each record in the `Character` array using data from each line of the file

As a reminder, the record structure is repeated here:

```
TYPE CharacterType
   DECLARE Player : INTEGER       //Field number 1
   DECLARE Role : STRING          //Field number 2
   DECLARE Name : STRING          //Field number 3
   DECLARE SkillLevel : INTEGER   //Field number 4
ENDTYPE
```

Write pseudocode for module `Restore()`.

You must use the module `Extract()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

BLANK PAGE

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character `'*'`.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
`"1234567890"`	`"******7890"`
`"1234567897652"`	`"*********7652"`
`"1234567890123456"`	`"************3456"`

(a) The function `Conceal()` will:

- take a numeric string as a parameter representing the card number
- return a string in which the asterisk character replaces all except the last four characters of the card number parameter.

Write pseudocode for the function `Conceal()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(i) Write pseudocode to declare the array.

.....

..... [2]

(ii) The new procedure `Conceal()` will write the modified string to the corresponding element in column two.

The array `CardNumber` is passed as a parameter to the new procedure `Conceal()`.

Identify how this parameter should be specified in the new procedure header.

.....

..... [1]

BLANK PAGE

4 A global integer variable `Tick` is always incremented every millisecond (1000 times per second) regardless of the other programs running.

The value of `Tick` can be read by any program but the value should not be changed.

Assume that the value of `Tick` does not overflow.

As an example, the following pseudocode algorithm would output `"Goodbye"` 40 seconds after outputting `"Hello"`.

```
DECLARE Start : INTEGER

OUTPUT "Hello"
Start ← Tick

REPEAT
   //do nothing
UNTIL Tick = Start + 40000

OUTPUT "Goodbye"
```

A program is needed to help a user to time an event such as boiling an egg.

The time taken for the event is known as the elapsed time.

The program contains a procedure `Timer()` which will:

- take two integer values representing an elapsed time in minutes and seconds
- use the value of variable `Tick` to calculate the elapsed time
- output a warning message 30 seconds before the elapsed time is up
- output a final message when the total time has elapsed.

For example, to set an alarm for 5 minutes and 45 seconds the program makes the following call:

```
CALL Timer(5, 45)
```

When 5 minutes and 15 seconds have elapsed, the program will output:

```
"30 seconds to go"
```

When 5 minutes and 45 seconds have elapsed, the program will output:

```
"The time is up!"
```

Write pseudocode for the procedure `Timer()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
   DECLARE Index, Count : INTEGER
   DECLARE ReturnValue : STRING

   Count ← INT(100 / Number)
   Index ← Data[Number]

   CASE OF (Index MOD 2)
      0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
      1 : ReturnValue ← "****"
   ENDCASE

   RETURN RetVal
ENDFUNCTION
```

(a) Run-time errors can be generated in different ways. For example, a run-time error will be generated if a function is called with invalid parameters.

The pseudocode contains three statements that could generate a run-time error.

Write the **three** statements **and** explain how each could generate a run-time error.

Statement 1 .....

Explanation .....

.....

Statement 2 .....

Explanation .....

.....

Statement 3 .....

Explanation .....

.....

[3]

(b) One type of run-time error can cause a program to stop responding ('freezing').

Identify a particular type of programming construct that can generate this type of error **and** explain why it occurs.

Construct .....

Explanation .....

.....

.....
[2]

(c) The function `Process()` contains a selection construct using a `CASE` structure.

Write pseudocode using a single selection construct with the same functionality **without** using a `CASE` structure.

.....

.....

.....

.....

..... [2]

6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	`"Cheese"`
2	`"Onion"`
3	`"Salmon"`
4	`"Anchovies"`
5	`"Peanut Butter"`

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	`"White"`
2	`"Brown"`
3	`"Pitta"`

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

(a) Complete the pseudocode for the procedure `Special()`.

Assume that both arrays are global.

```
PROCEDURE Special()
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDPROCEDURE
```

[7]

7 A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

(a) Identify **three** items of customer information that will be used by the new module **and** justify your choices.

Item 1 .....

Justification .....

.....

Item 2 .....

Justification .....

.....

Item 3 .....

Justification .....

.....

[3]

(d) Part of the coffee shop program contains three program modules as follows:

- Module `Init()` has no parameters and returns a Boolean.
- Module `Reset()` takes a string as a parameter and returns an Integer.
- Module `Check()` repeatedly calls `Init()` followed by `Reset()`.

Draw a structure chart to represent the relationship between the **three** modules, including all parameters and return values.

[3]

BLANK PAGE

8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Level	14, 23, 76	The skill level of the character. An integer in the range 0 to 100 inclusive.

The programmer has defined a record type to define each character.

The record type definition is shown in pseudocode as follows:

```
TYPE CharacterType
   DECLARE Player : INTEGER
   DECLARE Role : STRING
   DECLARE Name : STRING
   DECLARE Level : INTEGER
ENDTYPE
```

The `Player` field indicates the player to which the character is assigned (1 to 6). The field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data as follows:

```
DECLARE Character : ARRAY[1:45] OF CharacterType
```

At the start of the game all record fields are initialised, and all `Player` field values are set to 0

The programmer has defined a program module as follows:

Module	Description
`Assign()`	• called with two parameters: ○ an integer representing a player ○ a string representing a character role • search the `Character` array for an unassigned character with the required role • If found, assign the character to the given player and output a confirmation message, for example: `"Bill the Builder has been assigned to player 3"` • If no unassigned character with the required role is found, output a suitable message.

(a) Write pseudocode for module `Assign()`.

[7]

(b) A new module will store the contents of the `Character` array in a text file.

The module is defined as follows:

Module	Description
`Save()`	• form a string from each record with fields separated by the character `'^'` • write each string to a separate line of the new file named `SaveFile.txt`

Complete the pseudocode for module `Save()`.

```
PROCEDURE Save()
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing $100 would be $5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
   <= 50  : TaxRate ← 3.75      // tax rate of 3.75%
   <= 200 : TaxRate ← 5.23      // tax rate of 5.23%
   > 200  : TaxRate ← 6.25      // tax rate of 6.25%
            HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

(a) The pseudocode contains a logical error.

Identify the error **and** suggest a correction.

Error .....

.....

Correction .....

.....

[2]

(b) During the design of the program, tax rate values have been used wherever they are needed as shown in the pseudocode example above. Tax rates do not change while the program runs.

(i) Identify a more appropriate way of representing the tax rate values in the final program.

..... [1]

(ii) Describe the benefits of your answer to part **(b)(i)** with reference to this program.

.....

.....

.....

.....

.....

..... [3]

(d) The final `CASE` condition (`> 200`) in the pseudocode example could be replaced with a keyword.

Give the keyword.

..... [1]

2 A program uses three global integer variables `HH`, `MM` and `SS` to represent the current time in hours, minutes and seconds using the 24-hour clock notation.

Midnight would be represented as 00:00:00 (`HH`:`MM`:`SS`). If the variables `HH`, `MM` and `SS` contained the values 16, 30 and 10 respectively, then the time would be 16:30:10 or just after 4.30 in the afternoon.

A procedure `Tick()` will be called every second.
The procedure `Tick()` will:

- update the value in `SS` each time it is called
- update the values in `HH` and `MM` as appropriate
- call a procedure `CheckAlarm()` at the start of each minute
- call a procedure `NewDay()` whenever the time reaches midnight.

Complete the pseudocode for procedure `Tick()`.

```
PROCEDURE Tick()
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDPROCEDURE
```

[6]

4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

(a) Identify the type of error that could occur if these checks are **not** carried out **and** state a cause of this error.

Type .....

.....

Cause .....

.....

[2]

(b) The designer considers implementing the **checks and calculation** as a module (a procedure or a function). One reason for this is that the same checks and calculations are performed at several places in the program.

Give **another** reason why this is a suitable approach **and** state what is avoided by this approach.

Reason .....

.....

Avoided .....

.....

[2]

(c) The module to perform the checks and calculation will be implemented as a function. The function will need to return both a real **and** a Boolean value. To achieve this a record type is defined in pseudocode as follows:

```
TYPE Result
   DECLARE Done : BOOLEAN
   DECLARE Value : REAL
ENDTYPE
```

The function `Evaluate()` will:

- take two parameters of type string representing the two numeric values
- return a variable of type `Result` with the `Done` field set to `FALSE` if either of the following applies:
  - at least one of the strings does **not** represent a valid numeric value
  - the numeric value of the string representing value `y` is zero
- otherwise return a variable of type `Result` with the `Done` field set to `TRUE` and the `Value` field assigned the result of the formula (based on the numeric value of the two parameters).

Write pseudocode for the function `Evaluate()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

BLANK PAGE

6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
11	30
12	31

Note: Leap years are **not** considered

(a) An algorithm uses the array `DaysInMonth` to calculate a 'use-by' date. An alternative design would involve the use of multiple selection statements.

An array-based technique is often used when there is a large number of different values to check and where no pattern exists.

One advantage of using an array-based technique is the speed of execution compared to the use of multiple selection statements.

Give **two other** advantages of using an array for this type of operation rather than a solution based on multiple selection statements.

1 .....

.....

2 .....

.....

[2]

(b) Complete the pseudocode for the function `GetDate()`.

Date functions from the **insert** should be used in your solution.

```
FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

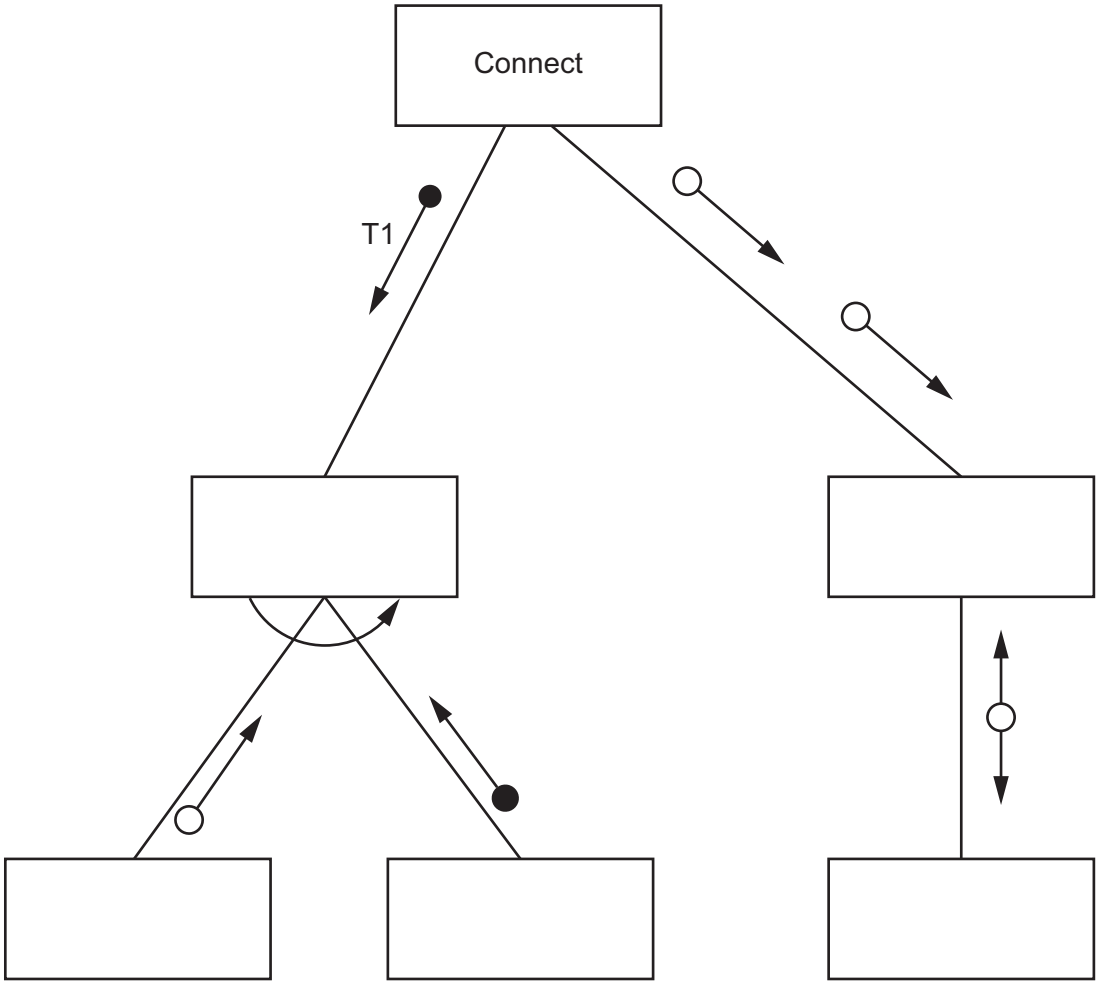
[7]

7 A program contains six modules with headers as follows:

Pseudocode module header
`PROCEDURE Connect()`
`FUNCTION Activate(H1 : STRING, Code : INTEGER) RETURNS BOOLEAN`
`FUNCTION Sync(T1 : BOOLEAN, S2 : REAL) RETURNS INTEGER`
`PROCEDURE Initialise(BYREF ID : INTEGER, BYVAL CC : INTEGER)`
`FUNCTION Reset(RA : STRING) RETURNS INTEGER`
`FUNCTION Enable(SA : INTEGER) RETURNS BOOLEAN`

Module `Connect()` will call either `Activate()` or `Sync()`. This is decided at run-time.

(a) Complete the structure chart for these modules.



[5]

BLANK PAGE

8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
`GradeBoundary[1]`	65	The minimum mark for an A grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark
`Result[4, 1]`	56
`Result[5, 1]`	54

Element	ID
`Result[4, 2]`	1074832
`Result[5, 2]`	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
`CheckMark()`	• called with a parameter of type integer representing a candidate mark • returns `TRUE` if the mark is within 2 of any of the five grade boundaries, otherwise returns `FALSE`

(a) Write pseudocode for module `CheckMark()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A second module is defined:

Module	Description
`CheckAll()`	• called with a parameter of type integer representing the number of candidate marks in the `Result` array • uses `CheckMark()` to check each candidate mark • for each paper that needs to be checked, write the corresponding candidate ID on a separate line in a new file named `GRList.txt` • outputs a message with a count of how many papers need to be checked

Write pseudocode for module `CheckAll()`.

`CheckMark()` must be used to check each individual mark.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program uses many complex algorithms.

One algorithm is repeated in several places. The code for the algorithm is the same wherever it is used, but the calculations within the algorithm may operate on different data.

The result of each calculation is used by the code that follows it.

It is decided to modify the program and implement the algorithm as a separate module.

(a) (i) State **two** benefits of this modification to the existing program.

1 .....

.....

2 .....

..... [2]

(ii) Describe how the modification would be implemented.

.....

.....

.....

.....

.....

.....

.....

..... [3]

(b) Four of the expressions used in the program are represented by pseudocode in the table.

Complete each pseudocode expression with a function or operator so that it evaluates to the value shown.

Any functions and operators used must be defined in the **insert**.

Pseudocode expression	Evaluates to
..... ("Random", 2, 3)	"and"
5 + ... (10/11/2023)	15
..... ("45000")	TRUE
(20 ..... 3) + 1	3

[4]

2 **(a)** A program uses a global 1D array of type string and a text file.

An algorithm that forms part of the program is expressed as follows:
- copy the first line from the file into the first element of the array
- copy the second line from the file into the second element of the array
- continue until all lines in the file have been copied into the array.

Stepwise refinement is applied to the algorithm.

Outline **five** steps for this algorithm that could be used to produce pseudocode.

Assume there are more elements in the array than lines in the file.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

[5]

(b) Sequence is one programming construct.

Identify **one other** programming construct that will be required when the algorithm from part **(a)** is converted into pseudocode **and** explain its use.

Construct .....

.....

Use .....

.....

[2]

4 A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:
- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

(a) Write pseudocode for the procedure `TwoParts()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 A program is being designed in pseudocode.

The program contains the following declaration:

```
DECLARE Data : ARRAY[1:1000] OF STRING
```

A procedure `ArrayInitialise()` is written to initialise the values in the array:

```
PROCEDURE ArrayInitialise(Label : STRING)
   DECLARE Index : INTEGER
   Index ← 1
   WHILE Index <= 1000
      CASE OF (Index MOD 2)
         0 : Data[Index] ← FormatA(Label)
             Index ← Index + 1
         1 : Data[Index] ← FormatB(Label)
             Index ← Index + 1
      ENDCASE
   ENDWHILE
ENDPROCEDURE
```

Functions `FormatA()` and `FormatB()` apply fixed format case changes to the parameter string.

(a) The design of the procedure does **not** use the most appropriate loop construct.

Suggest a **more appropriate** construct that could be used **and** explain your choice.

Construct .....

Explanation .....

.....

[2]

(b) The algorithm calls one of the functions `FormatA()` and `FormatB()` each time within the loop.

Explain why this is **not** efficient **and** suggest a more efficient solution.

.....

.....

.....

.....

.....

.....

.....

..... [4]

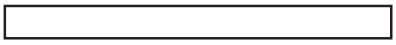
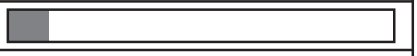
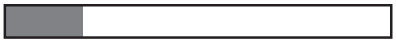
BLANK PAGE

6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	`BargraphA-1.bmp`	
2	>= 10 and < 20	`BargraphA-2.bmp`	
3	>= 20 and < 30	`BargraphA-3.bmp`	
9	>= 80 and < 90	`BargraphA-9.bmp`	
10	>= 90 and < 100	`BargraphA-10.bmp`	
11	100	`BargraphA-11.bmp`	

A procedure `Progress()` will:

- be called with two parameters:
  - an integer representing the percentage progress (0 to 100 inclusive)
  - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

(a) Write pseudocode for procedure `Progress()`.

[6]

(b) The definition of procedure `Progress()` is provided here for reference:

> A procedure `Progress()` will:
> - be called with two parameters:
> - an integer representing the percentage progress (0 to 100 inclusive)
> - a string representing the image filename root
> - generate the full image filename
> - call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

(i) Write pseudocode for the new module header.

.....

.....

..... [2]

BLANK PAGE

7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	• called with parameters Par1 and Par2 • calls either Mod-D or Mod-E, determined when the program runs • returns a Boolean value
Mod-C	• called with parameters Par1 and Par3 • Par3 is passed by reference • repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	• called with parameter Par3 • returns an integer value
Mod-F	called with parameter Par3
Mod-G	• called with parameter Par3 • Par3 is passed by reference

Parameters in the table are as follows:
- Par1 and Par3 are of type string.
- Par2 is of type integer.

(a) (i) Identify the modules that would be implemented as functions.

..... [1]

(ii) Modules Mod-F and Mod-G are both called with Par3 as a parameter.
In the case of Mod-F, the parameter is passed by value.
In the case of Mod-G, the parameter is passed by reference.

Explain the effect of the **two** different ways of passing the parameter Par3.

.....

.....

.....

..... [2]

(b) Draw a structure chart to show the relationship between the seven modules and the parameters passed between them.

[6]

8 A teacher is designing a program to process pseudocode projects written by her students.

The program analyses a student project and extracts information about each module that is defined (each procedure or function). This information is stored in a global 2D array `ModInfo` of type string.

A module header is the first line of a module definition and starts with either of the keywords `PROCEDURE` or `FUNCTION`.

An example of part of the array is given below. Row 10 of the array shows that a procedure header occurs on line 27 and row 11 shows that a function header occurs on line 35. `"P"` represents a procedure and `"F"` represents a function:

	x = 1	**x = 2**	**x = 3**
`ModInfo[10, x]`	`"27"`	`"P"`	`"MyProc(Z : CHAR)"`
`ModInfo[11, x]`	`"35"`	`"F"`	`"MyFun(Y : CHAR) RETURNS BOOLEAN"`

The string stored in column 3 is called the module description. This is the module header **without** the keyword.

A valid module header will:
- be at least 13 characters long
- start with the keyword `PROCEDURE` or `FUNCTION`. The keyword may appear in either upper or lower case (or a mix of both) and **must** be followed by a space character.

The teacher has defined the first program module as follows:

Module	Description

`Header()`	• called with a parameter of type string representing a line of pseudocode
	• if the line is a valid procedure header, returns a string:
	`"P<Module description>"`
	• if the line is a valid function header, returns a string:
	`"F<Module description>"`
	• otherwise, returns an empty string
	
	For example, given the string:
	
	`"FUNCTION Zap(X : INTEGER) RETURNS CHAR"`
	
	`Header()` returns the string:
	
	`"FZap(X : INTEGER) RETURNS CHAR"`

(a) Write pseudocode for module `Header()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A new module is required:

Module	Description
`FindModules()`	• called with a parameter of type string representing a student project file name • uses module `Header()` to check each line of the project • assigns values to the `ModInfo` array for each module declaration in the student project

As a reminder, the previous example of part of the array is repeated below:

	x = 1	**x = 2**	**x = 3**
`ModInfo[10, x]`	`"27"`	`"P"`	`"MyProc(Z : CHAR)"`
`ModInfo[11, x]`	`"35"`	`"F"`	`"MyFun(Y : CHAR) RETURNS BOOLEAN"`

Write pseudocode for module `FindModules()`.

Assume that the array contains enough rows for the number of modules in each project.

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:
- selection
- iteration (repetition)
- input/output.

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`			

`WRITEFILE ThisFile, "****"`			
`UNTIL Level > 25`			

`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`			

[4]

(b) Program variables have data types as follows:

Variable	Data type
`MyChar`	`CHAR`
`MyString`	`STRING`
`MyInt`	`INTEGER`

Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
`MyInt ← ..... (3.1415926)`
`MyChar ← ..... ("Elwood", 3, 1)`
`MyString ← ..... ( ..... (27.509))`
`MyInt ← ..... ( ..... ("ABC123", 3))`

[4]

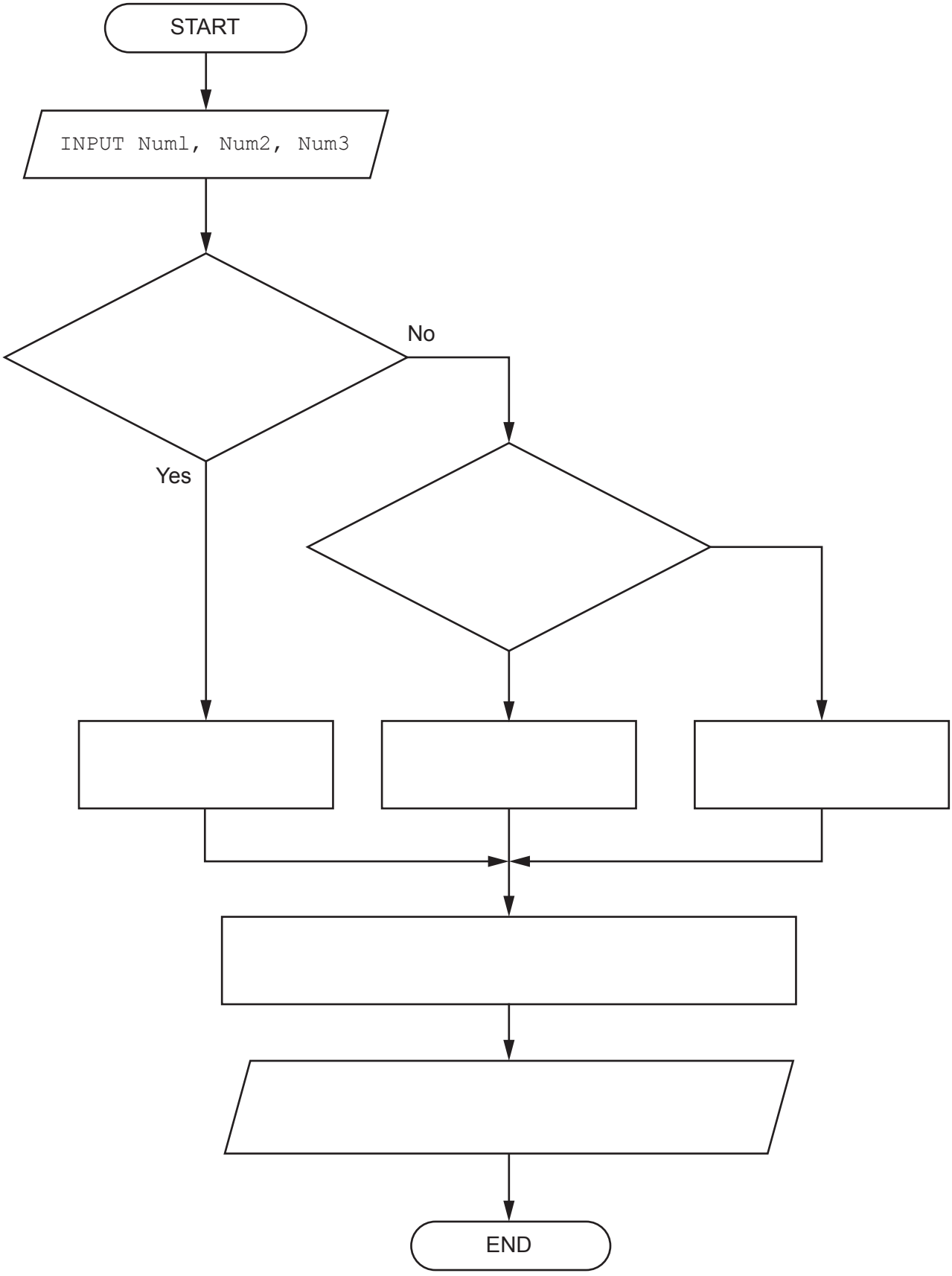
2 A program is being developed.

(a) An algorithm for part of the program will:

- input three numeric values and assign them to identifiers `Num1`, `Num2` and `Num3`
- assign the largest value to variable `Ans`
- output a message giving the largest value and the average of the three numeric values.

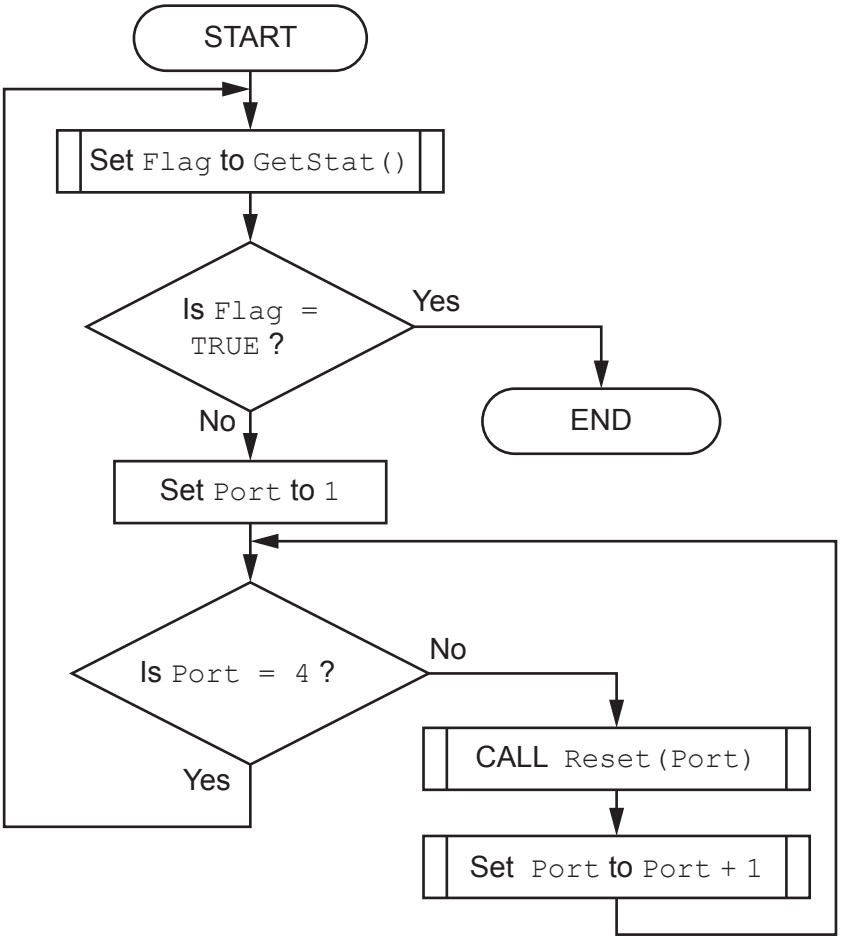
Assume the values are all different and are input in no particular order.

Complete the program flowchart on page 5 to represent the algorithm.



[5]

(b) A different part of the program contains an algorithm represented by the following program flowchart:



Write pseudocode for the algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

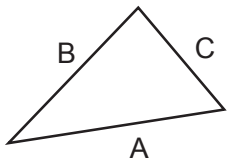
.....

.....

.....

..... [5]

4 A triangle has sides of length A, B and C.



In this example, A is the length of the longest side.

This triangle is said to be right-angled if the following equation is true:

$A \times A = (B \times B) + (C \times C)$

A procedure will be written to check whether three lengths represent a right-angled triangle. The lengths will be input in any sequence.

The procedure `IsRA()` will:

- prompt and input three integer values representing the three lengths
- test whether the three lengths correspond to the sides of a right-angled triangle
- output a suitable message.

The length of the longest side may **not** be the first value input.

Write pseudocode for the procedure `IsRA()`.

.....

.....

.....

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
    DECLARE Index : INTEGER
    Index ← 0
    INPUT Data[Index]
    WHILE Index < 200
        Index ← Index + 1
        CASE OF (Index MOD 2)
            0 : Data[Index] ← TO_UPPER(Label)
            1 : Data[Index] ← TO_LOWER(Label)
            OTHERWISE : OUTPUT "Alarm 1201"
        ENDCASE
    NEXT Index
    OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(a) (i) The pseudocode contains **two** syntax errors and **one** other error.

Identify the errors.

Syntax error 1 .....

.....

Syntax error 2 .....

.....

Other error .....

.....

[3]

(ii) The procedure contains a statement that is **not** needed.

Identify the pseudocode statement **and** explain why it is **not** needed.

Statement .....

Explanation .....

.....

[2]

6 A music player stores music in a digital form and has a display which shows the track being played.

(a) Up to 16 characters can be displayed. Track titles longer than 16 characters will need to be trimmed as follows:

- Words must be removed from the end of the track title until the resulting title is less than 14 characters.
- When a word is removed, the space in front of that word is also removed.
- Three dots are added to the end of the last word displayed when one or more words have been removed.

The table below shows some examples:

Original title	Display string															
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
Bat out of Hull	**B**	**a**	**t**		**o**	**u**	**t**		**o**	**f**		**H**	**u**	**l**	**l**	
Bohemian Symphony	**B**	**o**	**h**	**e**	**m**	**i**	**a**	**n**	**.**	**.**	**.**					
Paperbook Writer	**P**	**a**	**p**	**e**	**r**	**b**	**o**	**o**	**k**		**W**	**r**	**i**	**t**	**e**	**r**
Chris Sings the Blues	**C**	**h**	**r**	**i**	**s**		**S**	**i**	**n**	**g**	**s**	**.**	**.**	**.**		
Green Home Alabama	**G**	**r**	**e**	**e**	**n**		**H**	**o**	**m**	**e**	**.**	**.**	**.**			

A function `Trim()` will:

- take a string representing the original title
- return the string to be displayed.

Assume:

- Words in the original title are separated by a single space character.
- There are no spaces before the first word or after the last word of the original title.
- The first word of the original title is less than 14 characters.

Write pseudocode for the function `Trim()`.

.....

.....

.....

8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
`MichaelAday_src.txt`	Student project file produced by student Michael Aday
`MichaelAday_S1.txt`	File produced by stage 1
`MichaelAday_S2.txt`	File produced by stage 2

(a) Suggest a reason why the teacher's program has been split into a number of stages **and** give the benefit of producing a different file from each stage.

Reason .....

.....

Benefit .....

.....

[2]

(b) The teacher has defined the first program module as follows:

Module	Description
`DeleteSpaces()`	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: `"   IF X2 > 13 THEN"` After: `"IF X2 > 13 THEN"`

Complete the pseudocode for module `DeleteSpaces()`.

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[6]

(c) Two modules are defined:

Module	Description
`DeleteComment()` (already written)	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comment
`Stage_2()`	• called with two parameters: ○ a string representing an input file name ○ a string representing an output file name • copies each line from the input file to the existing output file having first removed all leading spaces and comments from that line • does not write blank lines to the output file • outputs a final message giving the number of blank lines removed

Write pseudocode for module `Stage_2()`.

Modules `DeleteComment()` and `DeleteSpaces()` must be used in your solution.

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 An algorithm is developed in pseudocode before being coded in a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables `A`, `B`, `C` and `D`.

Assignment statement	Data type
`A ← LEFT(MyName, 1)`	
`B ← Total * 2`	
`C ← INT(ItemCost) / 3`	
`D ← "Odd OR Even"`	

[4]

(b) Other variables in the program have example values as shown:

Variable	Value
`Sorted`	`False`
`Tries`	`9`
`ID`	`"ZGAC001"`

Complete the table by evaluating each expression, using the example values.

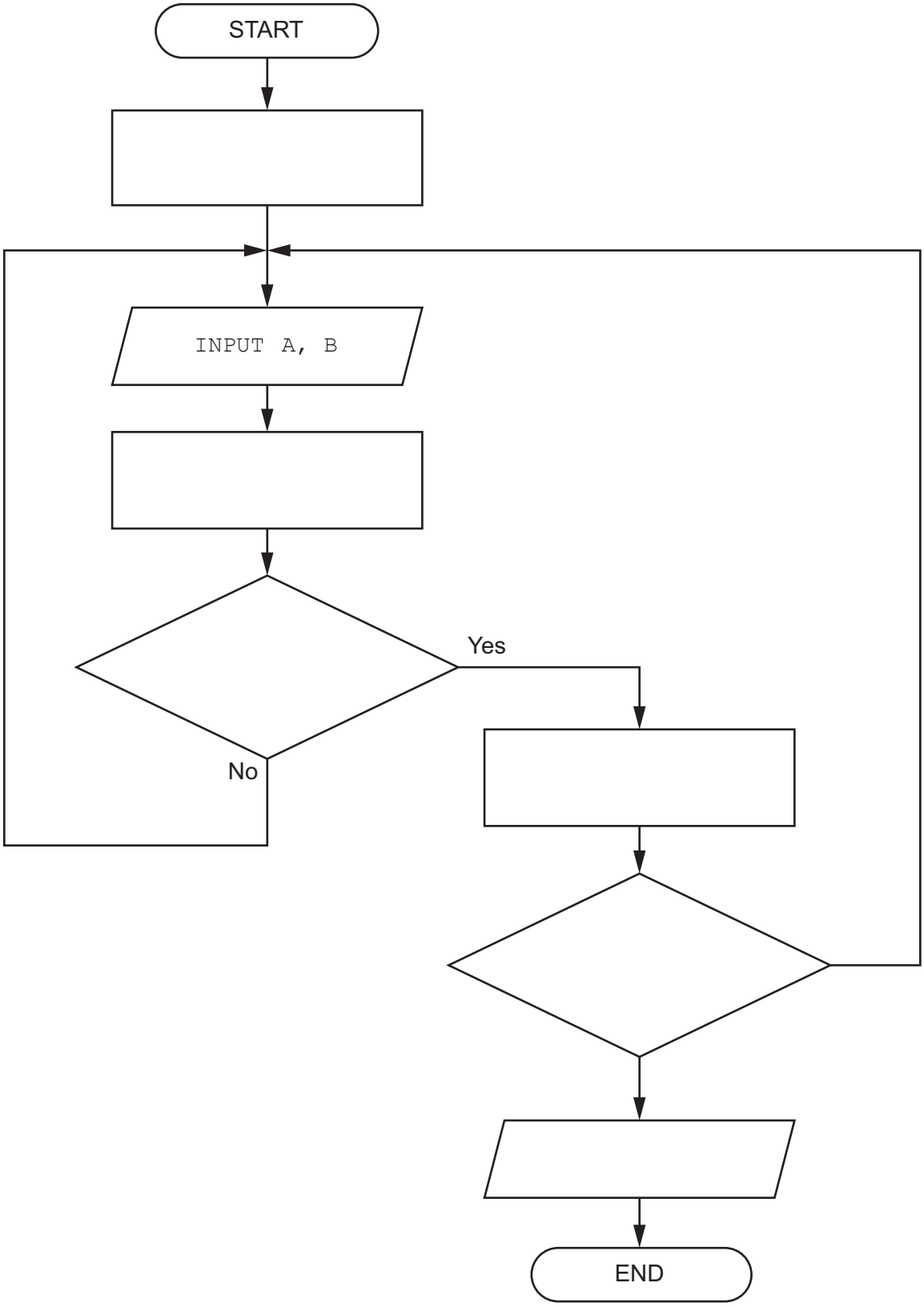
Expression	Evaluates to
`Tries < 10 AND NOT Sorted`	
`Tries MOD 4`	
`TO_LOWER(MID(ID, 3, 1))`	
`LENGTH(ID & "xx") >= Tries`	

[4]

2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values `A` and `B`
2. count the number of pairs that are input until `A` has been greater than `B` 10 times
3. output the number of pairs that were input.

(a) Complete the program flowchart.



[5]

(b) Step 1 of the algorithm is changed.

A variable `ThisSequence` is used to enter a sequence of 10 pairs of numeric values, using a single input statement.

Following the input of `ThisSequence` the revised algorithm will extract the pairs of numbers.

Describe the variable `ThisSequence` and how the numbers are extracted.

.....

.....

.....

..... [2]

4 A global 1D array `Data` contains 100 elements of type integer.

A function `Check()` will:

- total the element values in odd index locations (1, 3, 5 ... 97, 99)
- total the element values in even index locations (2, 4, 6 ... 98, 100)
- return one of three strings 'Odd', 'Even' or 'Same' to indicate which total is the greater, or whether the totals are the same.

Write pseudocode for the function `Check()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 A global 1D array of strings contains three elements which are assigned values as shown:

```
Data[1] ← "aaaaaa"
Data[2] ← "bbbbbb"
Data[3] ← "cccccc"
```

Procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Format : STRING)
   DECLARE Count, Index, L : INTEGER
   DECLARE Result : STRING
   DECLARE C : CHAR

   Result ← "****"

   FOR Count ← 1 TO LENGTH(Format) STEP 2
      C ← MID(Format, Count, 1)
      L ← STR_TO_NUM(MID(Format, Count + 1, 1))

      Index ← (Count + 1) DIV 2

      CASE OF C
         'X' : Result ← TO_UPPER(Data[Index])
         'Y' : Result ← TO_LOWER(Data[Index])
         'Z' : Result ← "**" & Data[Index]
      ENDCASE

      Data[Index] ← LEFT(Result, L)
   NEXT Count

ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called as follows:

```
CALL Process("X3Y2W4")
```

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]

[6]

(b) The procedure is to be modified. If variable `C` is assigned a value other than `'X'`, `'Y'` or `'Z'`, then procedure `Error()` is called and passed the value of variable `C` as a parameter.

This modification can be implemented by adding a **single line** of pseudocode.

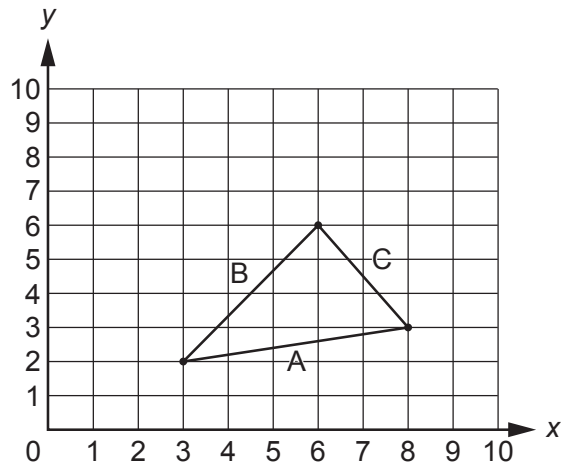
(i) Write the single line of pseudocode.

..... [1]

(ii) State where this new line should be placed.

..... [1]

6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



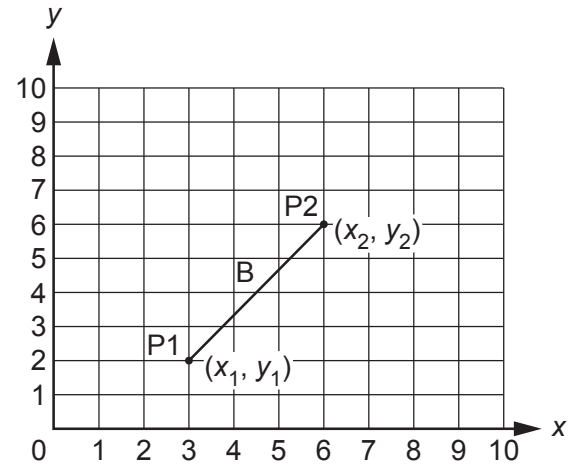
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$A^2 = B^2 + C^2$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2, B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$

In this example:

$B^2 = (3 - 6)^2 + (2 - 6)^2$
$B^2 = (-3)^2 + (-4)^2$
$B^2 = 9 + 16$
$B^2 = 25$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator '^' represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression $Value^2$ may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[6]

7 A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

(a) When an exercise class is planned, a new module will send personalised text messages to each member who has expressed an interest in that exercise. Members wishing to join the class send a text message back. Members may decide **not** to receive future text messages by replying with the message 'STOP'.

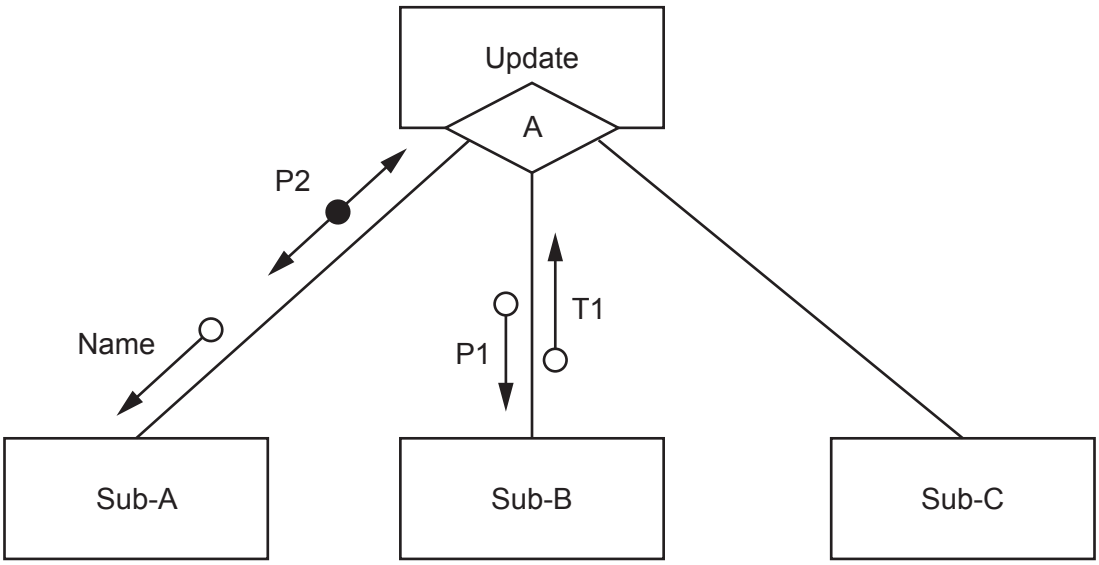
The process of abstraction is used to filter out unnecessary information.

(i) State **one** advantage of applying abstraction to this problem.

.....

..... [1]

(b) The structure chart illustrates part of the membership program:



Data item notes:

- Name contains the name of a club member
- P1 and T1 are of type real.

(i) Explain the meaning of the diamond symbol (labelled with the letter A) in the chart.

.....

.....

.....

..... [2]

(ii) Write the pseudocode module headers for Sub-A and Sub-B.

Sub-A

.....

.....

Sub-B

.....

.....

[4]

8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
`MichaelAday_src.txt`	student project file produced by student Michael Aday
`MichaelAday_S1.txt`	file produced by stage 1
`MichaelAday_S2.txt`	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
`DeleteComment()`	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: `IF X2 > 13 THEN   //check if limit exceeded` After: `IF X2 > 13 THEN`

(a) Complete the pseudocode for module `DeleteComment()`.

```
FUNCTION DeleteComment(Line : STRING) RETURNS STRING
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[8]

(b) A second module is defined:

Module	Description
`Stage_1()`	• called with a parameter of type string representing a student name • creates a new stage 1 file • copies each line from the student's project file to the stage 1 file after removing any comment from each line • does **not** write blank lines to the stage 1 file • returns the number of lines written to the stage 1 file

Write pseudocode for module `Stage_1()`.

Module `DeleteComment()` must be used in your solution.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed in pseudocode before being converted into a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving the data type that should be used to declare the variable underlined in each assignment statement.

Assignment statement	Data type
MyVar1 ← Total1 / Total2	
MyVar2 ← 27/10/2023	
MyVar3 ← "Sum1 / Sum2"	
MyVar4 ← Result1 AND Result2	

[4]

(b) Other variables in the program have example values as shown:

Variable	Value
Active	TRUE
Fraction	0.2
Code	"Ab12345"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
Fraction >= 0.2 AND NOT Active	
INT((Fraction * 100) + 13.3)	
STR_TO_NUM(MID(Code, 4, 2)) + 5	
LENGTH("TRUE" & Code)	

[4]

(c) The program makes use of complex statistical functions. The required functions are not built-in to the programming language and are too complicated for the programmer to write.

One solution would be to employ another programmer who has experience of writing these functions, as there is no time to train the existing programmer.

State **one other** way that these functions may be provided for inclusion in the program.

.....

..... [1]

4 A procedure `RandList()` will output a sequence of 25 random integers, where each integer is larger than the previous one.

(a) Write pseudocode for procedure `RandList()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) Procedure `RandList()` is modified so that the random numbers are also written into a 1D array `Result`.

A new module is written to confirm that the numbers in the array are in ascending order.

This module contains an `IF` statement that performs a comparison between elements:

```
IF (Result[x + 1] = Result[x]) OR (Result[x] > Result[x + 1]) THEN
   Sequence ← FALSE
ENDIF
```

Write a simplified version of the conditional clause.

.....

.....

..... [1]

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 4
Mix[2] ← 2
Mix[3] ← 3
Mix[4] ← 5
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Start : INTEGER)
   DECLARE Value, Index, Total : INTEGER

   Index ← Start
   Total ← 0

   WHILE Total < 20
      Value ← Mix[Index]
      Total ← Total + Value

      IF Index < 4 THEN
         Mix[Index] ← Mix[Index] + Mix[Index+1]
      ELSE
         Mix[Index] ← Mix[Index] + Mix[1]
      ENDIF
```

6 A function `TestNum()` will take a six-digit string as a parameter.

The function will test whether the string meets certain conditions and will return an integer value as follows:

Return value	Condition	Example
1	The last three digits are the same but non-zero.	"253444"
2	The last three digits are zero.	"253000"
3	The first three and last three digits are the same.	"410410"

The function will return the highest possible value for the given string.

If the string does not meet any of the conditions, zero is returned.

Write pseudocode for function `TestNum()`.

Assume that the parameter is valid.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

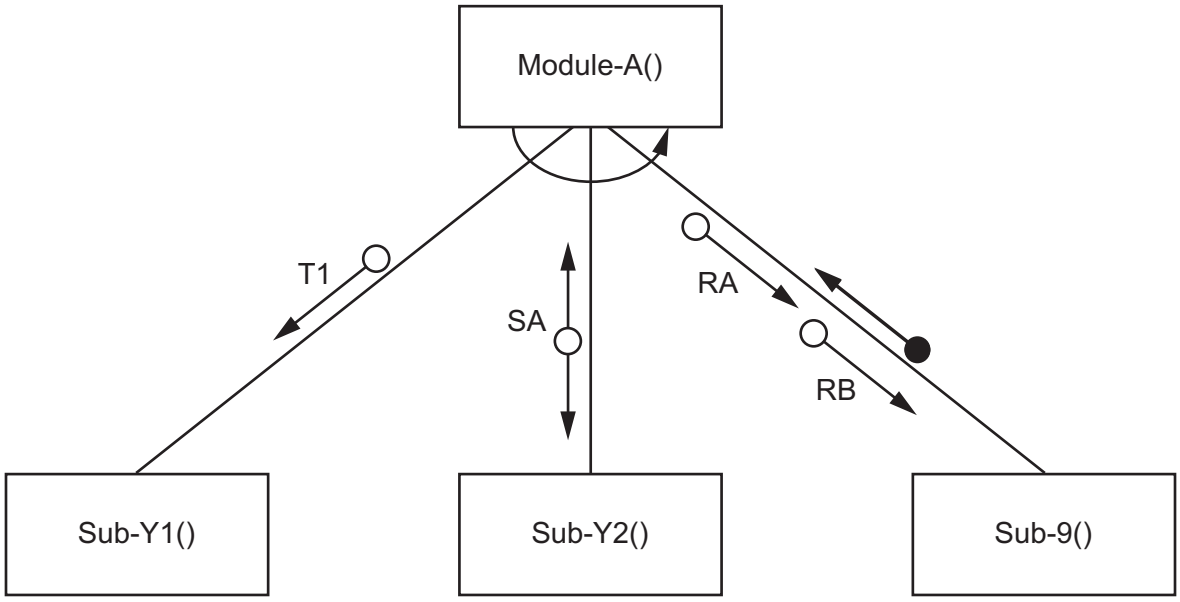
.....

.....

.....

..... [6]

7 A structure chart shows the modular structure of a program:



(a) Explain the meaning of the curved arrow symbol which begins and ends at Module-A().

.....

.....

.....

..... [2]

(b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

(i) Write pseudocode to define `MyType`.

.....

.....

.....

.....

.....

..... [3]

(ii) The design is modified and Sub-9() is changed to a procedure.

The procedure will be called with a single parameter of type `MyType`.

Write the pseudocode header for procedure `Sub-9()`.

.....

..... [2]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:
- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field name	Description
`STX`	a single character marking the start of the message (ASCII value 02)
`DestinationID`	three numeric characters identifying the destination computer
`SourceID`	three numeric characters identifying the source computer
`Data`	a variable length string containing the data being sent (Minimum length is 1 character)
`ETX`	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
`GetData()` (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
`ReceiveFile()`	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by `GetData()` to the file • repeats until the data field is "****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: `132456 characters were written to newfile.txt`

(a) Write pseudocode for module `ReceiveFile()`.

Module `GetData()` has already been written and must be used.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
`Transmit()` (already written)	• takes two parameters: ○ a string representing a message ○ an integer representing a port number • transmits the message using the given port
`Chat()`	• takes two parameters: ○ a string representing a Destination ID ○ an integer representing a port number • extracts data from a received message using `GetData()` and outputs it • forms a message using data input by the user and sends it using `Transmit()` • repeats until either the output string or the sent string is `"Bye"`

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

[7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
`"Mr Khan"`	The name of the customer		
`3`	The number of items in the order		
`TRUE`	To indicate whether this is a new customer		
`15.75`	The deposit paid by the customer		

[4]

(b) Other variables in the program have example values as shown:

Variable	Example value
`Total`	`124.00`
`DepRate`	`2.00`
`Description`	`"AB12345:Cleaning Brush (small)"`

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
`(Total * DepRate) + 1.5`	
`RIGHT(Description, 7)`	
`(LENGTH(Description) - 8) > 16`	
`NUM_TO_STR(INT(DepRate * 10)) & '%'`	

[4]

2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(a) Draw a program flowchart to represent the algorithm.



[5]

(b) The solution to the algorithm includes iteration.

Give the name of a suitable loop structure that could be used.

Justify your answer.

Name .....

Justification .....

.....

[2]

4 A procedure `Count()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(a) Write pseudocode for the procedure `Count()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

6 **(a)** A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, TestData.001, TestData.002 and TestData.003.

Each file will contain a single line. For example, file TestData.002 would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(i) Identify the type of module that should be used for `CheckFiles()`.

..... [1]

(ii) Write the module header for `CheckFiles()`.

.....

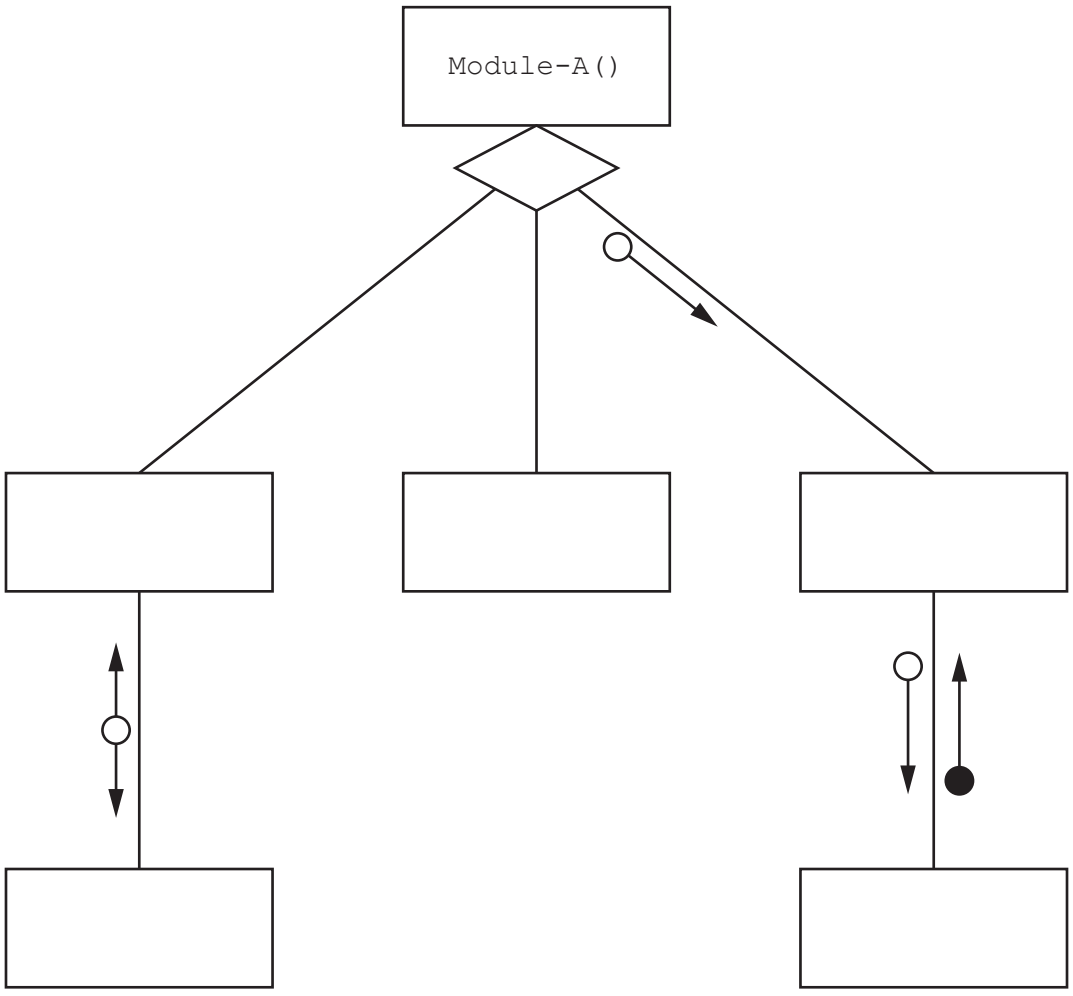
..... [1]

7 A program contains six modules:

Pseudocode module header
`PROCEDURE Module-A()`
`PROCEDURE Module-X(T1 : INTEGER, S2 : REAL)`
`PROCEDURE Reset(BYREF Code : INTEGER)`
`FUNCTION Restore(OldCode : INTEGER) RETURNS BOOLEAN`
`FUNCTION Module-Y(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN`
`FUNCTION Module-Z(SA : INTEGER) RETURNS INTEGER`

`Module-X()` calls `Reset()`
`Module-Y()` calls `Restore()`

(a) Complete the structure chart for these modules.



[4]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.
Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.
When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.
Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	**Column 2**	**Column 3**
Row 1	100	199	1
Row 2	200	259	2
Row 3	−1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to −1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
`GetPort()`	• takes a `DestinationID` as a parameter of type string • searches for the range corresponding to the `DestinationID` in the array • returns the port number, or returns −1 if no corresponding range is found

(a) Write pseudocode for module `GetPort()`.

Assume `DestinationID` contains a valid three-digit string.

..... [7]

(b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
`StackMsg()` (already written)	• takes two parameters: ○ a string representing a message ○ an integer representing the stack number • adds the message to the required stack • returns `TRUE` if the message is added to the required stack, otherwise returns `FALSE`
`ProcessMsg()`	• takes a message as a parameter of type string • ignores any message with a zero-length data field • extract the `DestinationID` from the message • checks whether the `DestinationID` is this computer or whether the message is to be forwarded • uses `StackMsg()` to add the message to the appropriate stack • outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 The following pseudocode represents part of the algorithm for a program:

```
CASE OF ThisValue
   < 30       : Level ← "Low"           // less than 30
                Check ← 1
   < 20       : Level ← "Very Low"      // less than 20
                Check ← ThisValue / 2
   30 TO 40   : Level ← "Medium"        // between 30 and 40
                Check ← ThisValue / 3
                Data[ThisValue] ← Data[ThisValue] + 1
   > 40       : Level ← "High"
ENDCASE
```

(a) Complete the table by writing the answer for each row:

	Answer
The value assigned to `Level` when `ThisValue` is 40	
The value assigned to `Check` when `ThisValue` is 36	
The value assigned to `Level` when `ThisValue` is 18	
The number of elements in array `Data` that may be incremented	

[4]

(b) The pseudocode contains four assignments to variable `Level`. One of these assignments will never be performed.

Identify this assignment **and** explain why this is the case.

.....

.....

.....

.....

.....

..... [3]

(c) The following line is added immediately before the `ENDCASE` statement:

```
OTHERWISE : Level ← "Undefined"
```

State why this assignment is never performed.

.....

..... [1]

2 **(a)** An algorithm is expressed as follows:

- input 100 numbers, one at a time
- keep a total of all numbers input that have a value between 30 and 70 inclusive and output this total after the last number has been input.

Outline, using stepwise refinement, the five steps for this algorithm which could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

[5]

(b) Sequence is one programming construct.

Identify **two other** programming constructs that will be required when the algorithm is converted into pseudocode.

Construct 1 .....

.....

Construct 2 .....

.....

[2]

4 A global array is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150] OF STRING
```

A function `TooMany()` will:

1. take two parameters:
   - a string (the search string)
   - an integer (the maximum value)
2. count the number of strings in the array that exactly match the search string
3. return `TRUE` if the count is greater than the maximum value, otherwise will return `FALSE`

(a) Write pseudocode for the function `TooMany()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The global array is changed to a 2D array, organised as 150 rows by 2 columns. It is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150, 1:2] OF STRING
```

The algorithm for the function in **part (a)** is changed. Strings will only be counted if **both** of the following conditions are true:

- The current row is an even number.
- The search string exactly matches the value in **either** column.

Write pseudocode to check these conditions.

Assume that the row index is contained in variable `Row` and the search string in variable `Search`.

.....

.....

.....

.....

.....

..... [3]

5 An algorithm is designed to find the smallest numeric value from an input sequence and count how many numeric values have been input.
An example of an input sequence is:

23, AB56, 17, 23ZW, 4, 10, END

Numeric input values are all integers and non-numeric input is ignored, except for the string "END" which is used to terminate the sequence.

The algorithm is expressed in pseudocode as shown:

```
DECLARE NextInput : STRING
DECLARE Min, Count, Num : INTEGER

Min ← 999
Count ← 0

REPEAT
   INPUT NextInput
   IF IS_NUM(NextInput) = TRUE THEN
      Num ← STR_TO_NUM(NextInput)
      IF Num > Min THEN
         Min ← Num
      ENDIF
      Count ← Count & 1
   ENDIF
UNTIL NextInput ← "END"

OUTPUT "The minimum value is ", Min, " and the count was ", Count
```

(a) The pseudocode contains three errors due to the incorrect use of **operators**.

Identify each error **and** state the correction required.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

6 The pseudocode `OUTPUT` command starts each output on a new line.

(a) A new procedure `MyOutput()` will take a string and a Boolean parameter. `MyOutput()` may be called repeatedly and will use concatenation to build a string using a global variable `MyString`, up to a maximum length of 255 characters.

`MyString` will be output in either of these two cases:

1. The Boolean parameter value is `TRUE`
2. The resulting string (after concatenation) would be longer than 255 characters.

If `MyString` is not output, the string is concatenated with `MyString`.

For example, the calls to `MyOutput()` given below would result in the output as shown:

```
MyOutput("Hello ", FALSE)
MyOutput("ginger ", FALSE)
MyOutput("cat", TRUE)
MyOutput("How are you?", TRUE)
```

Resulting output:

```
Hello ginger cat
How are you?
```

Notes:

- `MyString` is initialised to an empty string before `MyOutput()` is called for the first time.
- No string passed to `MyOutput()` will be longer than 255 characters.

Write pseudocode for `MyOutput()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) The design of the procedure given in **part (a)** is modified and `MyString` is changed from a global to a local variable declared in `MyOutput()`.

When the modified procedure is converted into program code, it does not work as expected.

Explain why it does not work as expected.

.....

.....

.....

..... [2]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field number	Field name	Description
n/a	`STX`	a single character marking the start of the message (ASCII value 02)
1	`DestinationID`	three numeric characters that identify the destination computer
2	`SourceID`	three numeric characters that identify the source computer
3	`Data`	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	`ETX`	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
`Transmit()` (already written)	• takes two parameters: ○ a string containing a message ○ an integer containing a port number • transmits the message using the given port
`SendFile()`	• takes three parameters: ○ a string containing a text file name ○ a string containing a Destination ID ○ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "`****`"

(a) Write pseudocode for module `SendFile()`.

Assume:

- module `Transmit()` has already been written and is used to transmit a message
- the value of `MyID` may be used as `SourceID`
- the file specified contains no blank lines
- the file specified does not contain the line "`****`"

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

BLANK PAGE

Question 8(d) starts on page 18.

(d) A new module has been defined:

Module	Description
`GetField()`	• takes two parameters: ○ a string containing a message ○ an integer containing a field number • If the field number is valid (in the range 1 to 3, inclusive), it returns a string containing the required field, otherwise it returns an empty string.

As a reminder, a message is defined as follows:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field number	Field name	Description
Not applicable	`STX`	a single character marking the start of the message (ASCII value 02)
1	`DestinationID`	three numeric characters that identify the destination computer
2	`SourceID`	three numeric characters that identify the source computer
3	`Data`	a variable length string containing the data being sent (Minimum length is 1 character)
Not applicable	`ETX`	a single character marking the end of the message (ASCII value 03)

Write pseudocode for module `GetField()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

Refer to the **insert** for the list of pseudocode functions and operators.

1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```
10   DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
…
100  FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103  NEXT PCount
```

(a) The pseudocode contains references to an array.

Complete the table by writing the answer for each row.

	Answer
The dimension of the array	
The name of the variable used as an array index	
The number of elements in the array	

[3]

(d) The pseudocode statements given in the following table are used in other parts of the algorithm.

Complete the table by placing **one or more** ticks (✓) in each row.

The first row has already been completed.

Pseudocode statement	**Input**	**Process**	**Output**
`INPUT MyChoice`	✓		
`OUTPUT FirstName & LastName`			
`WRITEFILE YourFile, TextLine`			
`READFILE MyFile, TextLine`			
`Result ← SQRT(NextNum)`			

[4]

2 A program stores a date of birth for a student using a variable, `MyDOB`, of type `DATE`.

(a) `MyDOB` has been assigned a valid value corresponding to Kevin's date of birth.

Complete the pseudocode statement to test whether Kevin was born on a Thursday.

`IF` ..... `THEN` [2]

4 A function `MakeString()` will:

1. take two parameters:
   - a count as an integer
   - a character
2. generate a string of length equal to the count, made up of the character
3. return the string generated, or return `"ERROR"` if the count is less than 1.

For example, the function call:

`MakeString(3, 'Z')` will return the string `"ZZZ"`

Write pseudocode for function `MakeString()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

6 A procedure `Select()` will:

1. take two integer values as parameters representing start and end values where both values are greater than 9 and the end value is greater than the start value
2. output **each** integer value between the start and the end value (**not** including the start and end values), where the sum of the last two digits is 6, for example, 142.

(a) Write pseudocode for procedure `Select()`.

Parameter validation is **not** required.

..... [7]

(b) The check performed by procedure `Select()` on the last two digits is needed at several places in the program and will be implemented using a new function.

The new function `CheckNum()` will:

- allow the required sum to be specified (not just 6)
- check one number
- return an appropriate value.

Describe the function interface **and two** advantages of this modular approach.

Interface .....

.....

.....

.....

Advantage 1 .....

.....

Advantage 2 .....

.....

[4]

7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

A program helps the staff to manage the borrowing of books.

(a) A new module needs to be written to generate emails to send to students who have an overdue book. Students who are sent an email are prevented from borrowing any more books until the overdue book is returned.

The process of abstraction has been used when designing the new module.

(i) State the purpose of applying abstraction to this problem.

.....

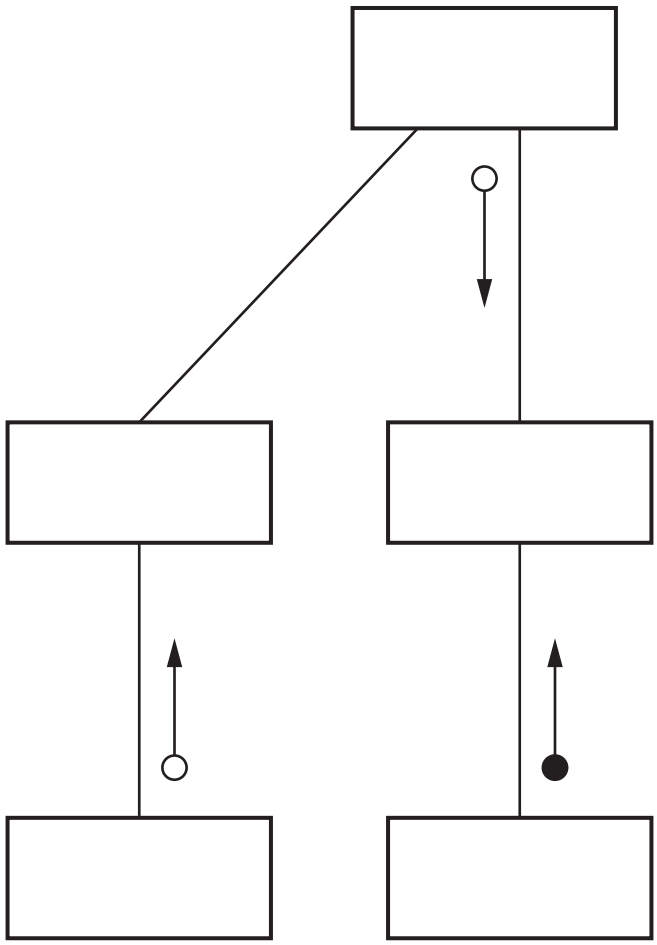
..... [1]

(b) Part of the library program contains program modules with headers as follows:

Pseudocode module header
`PROCEDURE Module-X()`
`PROCEDURE Module-Y(BYREF RA : INTEGER, SA : REAL)`
`PROCEDURE Overlay()`
`FUNCTION Replace(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN`
`FUNCTION Reset(TA : STRING) RETURNS INTEGER`

`Module-X()` and `Module-Y()` are both called from module `Overlay()`.

Complete the structure chart.



[3]

8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
`SupplierCode`	3 alphabetic characters	code to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
`ChangeSupp()`	• called with two parameters `Code1` and `Code2` of type string that represent valid supplier codes • creates a new file `NewStock.txt` from the contents of the file `Stock.txt` where any reference to `Code1` is replaced by `Code2` • returns a count of the number of items that have had their supplier code changed

(a) Write pseudocode for module `ChangeSupp()`.

[8]

(b) A new module is required:

Module	Description

`Report_1()`	• takes a parameter of type string that represents a `SupplierCode`
	• searches the `Stock.txt` file for each line of item information that contains the given `SupplierCode`
	• produces a formatted report of items for the given `SupplierCode`, for example, for supplier DRG, the output could be:
	
	`Report for Supplier: DRG`
	
	`Item  Description`
	
	`1234  USB Printer Cable 3m`
	`1273  32GB USB Flash Drive`
	`1350  Mouse Mat 320 x 240mm`
	
	`Number of items listed: 3`

Write pseudocode for module `Report_1()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of $3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
   ItemPostalCost ← 3.75    // set postal cost for item to $3.75
   ItemStatus ← "Valid"     // item can be sent
ENDIF
```

(a) (i) Identify a more appropriate way of representing the postal costs.

..... [1]

(ii) Describe the advantages of your answer to **part (a)(i)** with reference to this program.

.....

.....

.....

.....

.....

..... [3]

2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

(a) Write a pseudocode statement, using a function from the **insert**, to assign the value corresponding to 17/11/2007 to `MyDOB`.

..... [1]

(b) `MyDOB` has been assigned a valid value representing the user's date of birth.

Write a pseudocode statement to calculate the number of months from the month of the user's birth until the end of the year and to assign this to the variable `NumMonths`.

For example, if `MyDOB` contains a value representing 02/07/2008, the value 5 would be assigned to `NumMonths`.

..... [2]

(c) The program will output the day of the week corresponding to `MyDOB`.

For example, given the date 22/06/2023, the program will output `"Thursday"`.

An algorithm is required. An array will be used to store the names of the days of the week.

Define the array **and** describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Array definition .....

.....

Step 1 .....

.....

.....

Step 2 .....

.....

.....

Step 3 .....

.....

.....

Step 4 .....

.....

.....

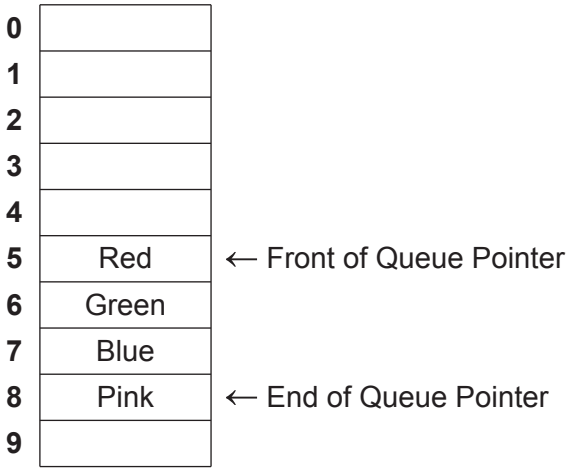
[6]

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

(a) The diagram below represents an Abstract Data Type (ADT) implementation of the queue. Each data item is stored in a separate location in the data structure. During initial design, the queue is limited to holding a maximum of 10 data items.

The operation of this queue may be summarised as follows:

- The Front of Queue Pointer points to the next data item to be removed.
- The End of Queue Pointer points to the last data item added.
- The queue is circular so that locations can be reused.



(i) Describe how the data items Orange and Yellow are added to the queue shown in the diagram.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(b) The design of the queue is completed and the number of locations is increased.

A function `AddToQueue()` has been written. It takes a string as a parameter and adds this to the queue. The function will return `TRUE` if the string was added successfully.

A procedure `FileToQueue()` will add each line from the file to the queue. This procedure will end when all lines have been added or when the queue is full.

Describe the algorithm for procedure `FileToQueue()`.

Do **not** use pseudocode in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

4 A function `GetNum()` will:

1. take two parameters: a string and a character
2. count the number of times that the character occurs in the string
3. return the count.

Any comparison between characters needs to be case sensitive. For example, character '`a`' and character '`A`' are not identical.

Write pseudocode for function `GetNum()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 A programmer has produced the following pseudocode to output the square root of the numbers from 1 to 10.

Line numbers are for reference only.

```
10  DECLARE Num : REAL
11  Num ← 1.0
...
40  REPEAT
41     CALL DisplaySqrt(Num)
42     Num ← Num + 1.0
43  UNTIL Num > 10
...
50  PROCEDURE DisplaySqrt(BYREF ThisNum : REAL)
51     OUTPUT ThisNum
52     ThisNum ← SQRT(ThisNum)       // SQRT returns the square root
53     OUTPUT " has a square root of ", ThisNum
54  ENDPROCEDURE
```

The pseudocode is correctly converted into program code.

Function `SQRT()` is a library function and contains no errors.

The program code compiles without errors, but the program gives unexpected results. These are caused by a design error.

(a) Explain why the program gives unexpected results.

.....

.....

.....

.....

.....

..... [3]

6 A procedure `Square()` will take an integer value in the range 1 to 9 as a parameter and output a number square.

The boundary of a number square is made up of the character representing the parameter value. The inside of the number square is made up of the asterisk character (`*`).

Parameter value	**1**	**2**	**3**	**4**	**...**	**9**
Output	1	22 22	333 3*3 333	4444 4**4 4**4 4444	...	999999999 9*******9 9*******9 9*******9 9*******9 9*******9 9*******9 9*******9 999999999

The pseudocode `OUTPUT` command starts each output on a new line. For example, the following three `OUTPUT` statements would result in the outputs as shown:

```
OUTPUT "Hello"
OUTPUT "ginger"
OUTPUT "cat"
```

Resulting output:

```
Hello
ginger
cat
```

Write pseudocode for procedure `Square()`.

Parameter validation is not required.

.....

.....

.....

.....

.....

..... [6]

8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Valid item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique number for each item in the range ″0001″ to ″5999″ inclusive
`SupplierCode`	3 alphabetic characters	to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
`OnlyAlpha()` (already written)	• called with a parameter of type string • returns `TRUE` if the string contains only alphabetic characters, otherwise returns `FALSE`
`CheckInfo()`	• called with a parameter of type string representing a line of item information • checks to see whether the item information in the string is valid • returns `TRUE` if the item information is valid, otherwise returns `FALSE`

(a) Write pseudocode for module `CheckInfo()`.

Module `OnlyAlpha()` should be used as part of your solution.

.....

.....

.....

..... [7]

(b) A new module is defined as follows:

Module	Description
`AddItem()`	• called with a parameter of type string representing valid information for a new item that is not currently in the `Stock.txt` file • creates a new file `NewStock.txt` from the contents of the file `Stock.txt` and adds the new item information at the appropriate place in the `NewStock.txt` file

As a reminder, the file `Stock.txt` is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

Write pseudocode for module `AddItem()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A programmer has written a program which includes the function `Calculate()`.
When the program is run, the function returns an unexpected value.

(a) Describe how a typical Integrated Development Environment (IDE) could be used to help debug the program to find the errors in the function `Calculate()`.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(b) The algorithm for function `Calculate()` contains the three pseudocode statements shown.

Describe the error in each statement or write 'no error' if the statement contains no error.

Assume any variables used are of the correct type for the given function.

Statement 1: `Index ← STR_TO_NUM(("27") + 2)`

Error .....

.....

Statement 2: `Index ← STR_TO_NUM(MID("CPE1704TKS", 4, 2))`

Error .....

.....

Statement 3: `IF MONTH(ThisDate) > '6' THEN`

Error .....

.....
[3]

(c) The program contains variables with values as follows:

Variable	Value
Active	TRUE
Points	75
Exempt	FALSE

(i) Complete the table by evaluating each expression.

	Expression	Evaluation
1	(Points > 99) OR Active	
2	(Points MOD 2 = 0) OR Exempt	
3	(Points <= 75) AND (Active OR Exempt)	
4	(Active OR NOT Active) AND NOT Exempt	

[2]

(ii) Write expression **4** from the table in part **(c)(i)** in its simplest form.

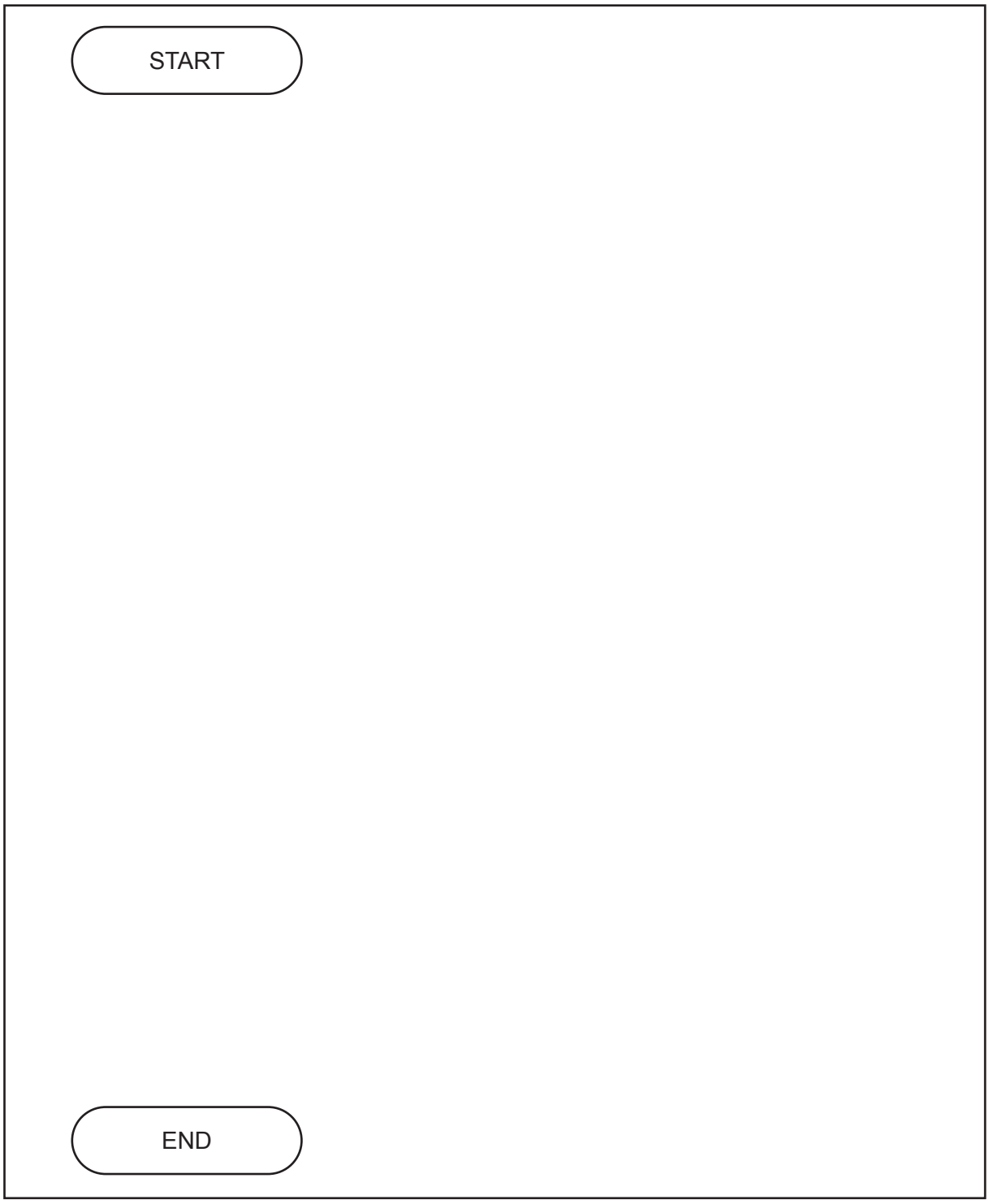
..... [1]

2 A program contains an algorithm to output a string of a specified length containing identical characters.

(a) The algorithm is described as follows:

1. prompt and input a character and store in `MyChar`
2. prompt and input an integer and store in `MyCount`
3. generate a string consisting of `MyChar` repeated `MyCount` times
4. output the string.

Draw a program flowchart to represent the algorithm.



[4]

(b) A different part of the program uses the variable `StartDate`.

Write pseudocode statements to declare `StartDate` **and** assign to it the date corresponding to 15/11/2005.

Declaration .....

Assignment .....

[3]

4 Function `Replace()` will:

1. take three parameters:
   - a string (the original string)
   - a char (the original character)
   - a char (the new character)
2. form a new string from the original string where all instances of the original character are replaced by the new character
3. return the new string.

Write pseudocode for function `Replace()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

6 A video-conferencing program supports up to six users. Speech from each user is sampled and digitised (converted from analogue to digital). Digitised values are stored in array `Sample`.

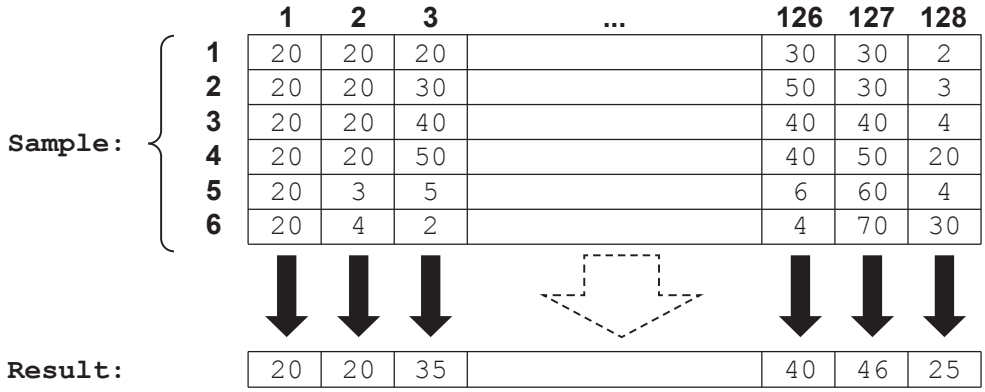
The array `Sample` consists of 6 rows by 128 columns and is of type integer. Each row contains 128 digitised sound samples from one user.

The digitised sound samples from each user are to be processed to produce a single value which will be stored in a 1D array `Result` of type integer. This process will be implemented by procedure `Mix()`.

A procedure `Mix()` will:

- calculate the average of each of the 6 sound samples in a column
- ignore sound sample values of 10 or less
- store the average value in the corresponding position in `Result`
- repeat for each column in array `Sample`

The diagram uses example values to illustrate the process:



Write pseudocode for procedure `Mix()`.

Assume `Sample` and `Result` are global.

..... [6]

8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
`SupplierCode`	5 alphabetic characters	to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
`SuppExists()` (already written)	• called with a parameter of type string representing a supplier code • returns `TRUE` if the supplier code is already in use, otherwise returns `FALSE`
`IsNewSupp()`	• called with a parameter of type string representing a new supplier code • returns `TRUE` if the string only contains alphabetic characters (either upper or lower case) and the supplier code is **not** already in use, otherwise returns `FALSE`

(a) Write pseudocode for module `IsNewSupp()`.

Module `SuppExists()` has already been written and should be used as part of your solution. Module `SuppExists()` will generate a run-time error if the given parameter is not 5 characters in length.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A new module has been defined:

Module	Description
`CheckNewItem()`	• called with a parameter of type string representing a line of item information • checks to see whether an item with the same `ItemNum` already exists in the file • returns `TRUE` if the `ItemNum` is not already in the file, otherwise returns `FALSE`

Write **efficient** pseudocode for module `CheckNewItem()`.

.....

.....

.....

..... [7]

(e) A new module `FindItem()` searches for a given item in the `Stock.txt` file, which is already organised in ascending order of `ItemNum`.

Describe how this organisation may improve the efficiency of the algorithm.

.....

.....

.....

.....

.....

..... [3]

4 **(a)** A program contains a 1D array `DataItem` with 100 elements.

State the **one additional** piece of information required before the array can be declared.

.....

..... [1]

(b) A programmer decides to implement a queue Abstract Data Type (ADT) in order to store characters received from the keyboard. The queue will need to store at least 10 characters and will be implemented using an array.

(i) Describe **two** operations that are typically required when implementing a queue. State the check that must be carried out before each operation can be completed.

Operation 1 .....

.....

Check 1 .....

.....

Operation 2 .....

.....

Check 2 .....

.....

[4]

(ii) Describe the declaration and initialisation of the variables and data structures used to implement the queue.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

5 **(a)** A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

(i) Write pseudocode for the procedure `Unpack()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(ii) Explain the term **procedure interface** with reference to procedure `Unpack()`.

.....

.....

.....

..... [2]

(b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
   DECLARE StockID : STRING
   DECLARE Description : STRING
   DECLARE Cost : REAL
ENDTYPE
```

(i) A variable `LineData` of type `StockItem` is declared.

Write the pseudocode statement to assign the value `12.99` to the `Cost` field of `LineData`.

..... [1]

(ii) Procedure `Unpack()` is modified and converted to a function which takes the original text string as the only parameter.

Explain the other changes that need to be made to convert the procedure into a function.

.....

.....

.....

.....

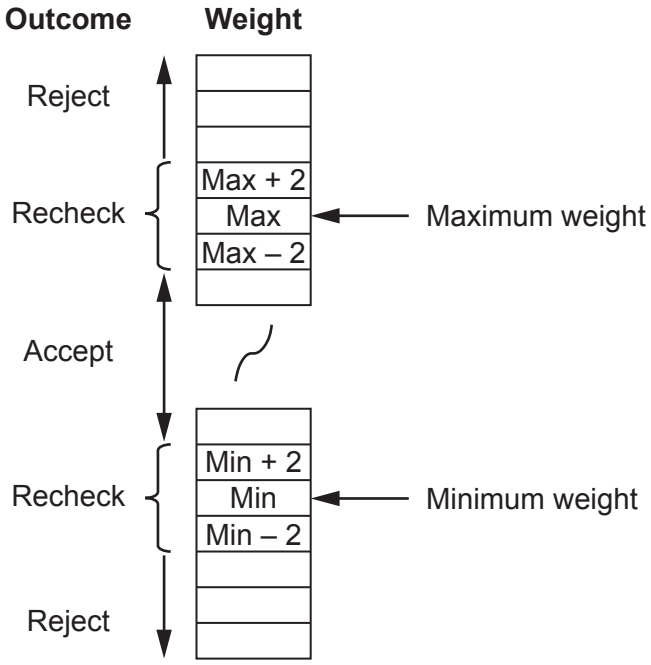
..... [2]

6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	`'A'`
Reject	`'R'`
Recheck	`'C'`

(a) Complete the following test plan for **five** tests that could be performed on function `Status()`. The tests should address all possible outcomes.

Test number	Component weight	Min	Max	Expected return value
1				`'A'`
2				
3				
4				
5				

[5]

(b) Write pseudocode for `Status()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

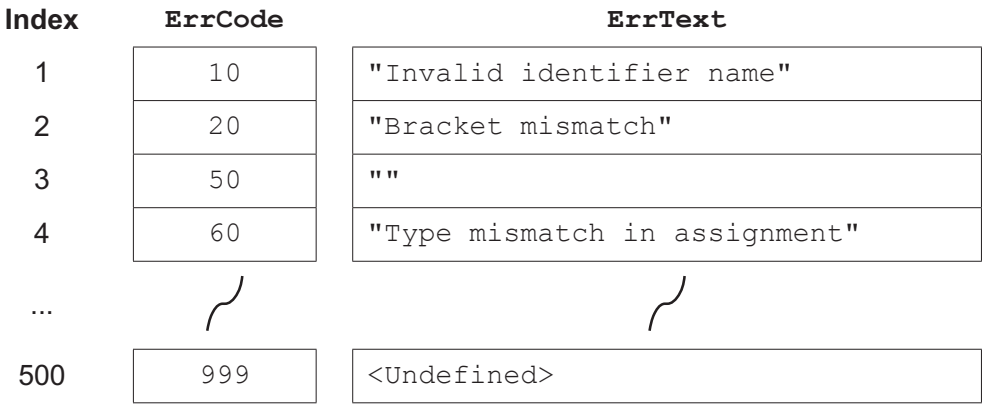
..... [6]

7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.



Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description

`OutputRange()`	• Prompts for input of two error numbers
	• Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description
	• Outputs a warning message when the error description is missing as for error number 50 in the example
	• Outputs a suitable header and a final count of error numbers found
	
	Output based on the example array data above:
	
	`List of error numbers from 1 to 60`
	
	`10 : Invalid identifier name`
	`20 : Bracket mismatch`
	`50 : Error Text Missing`
	`60 : Type mismatch in assignment`
	
	`4 error numbers output`

(a) Write pseudocode for module `OutputRange()`. Assume that the two numbers input represent a valid error number range.

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

Question 7 continues on the next page.

(b) (i) Two additional modules are defined:

Module	Description
`SortArrays()`	• Sorts the arrays into ascending order of `ErrCode`
`AddError()`	• Takes two parameters: ○ an error number as an integer ○ an error description as a string • Writes the error number and error description to the first unused element of the two arrays. Ensures the `ErrCode` array is still in ascending order • Returns the number of unused elements **after** the new error number has been added • Returns −1 if the new error number could not be added

Write pseudocode for the module `AddError()`. Assume that the error code is **not** already in the `ErrCode` array.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1 .....

.....

2 .....

.....

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc(Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers `Count` and `Message` **and** explain their use.

Term .....

Use .....

.....

.....

.....

[2]

(c) Part of an identifier table is shown:

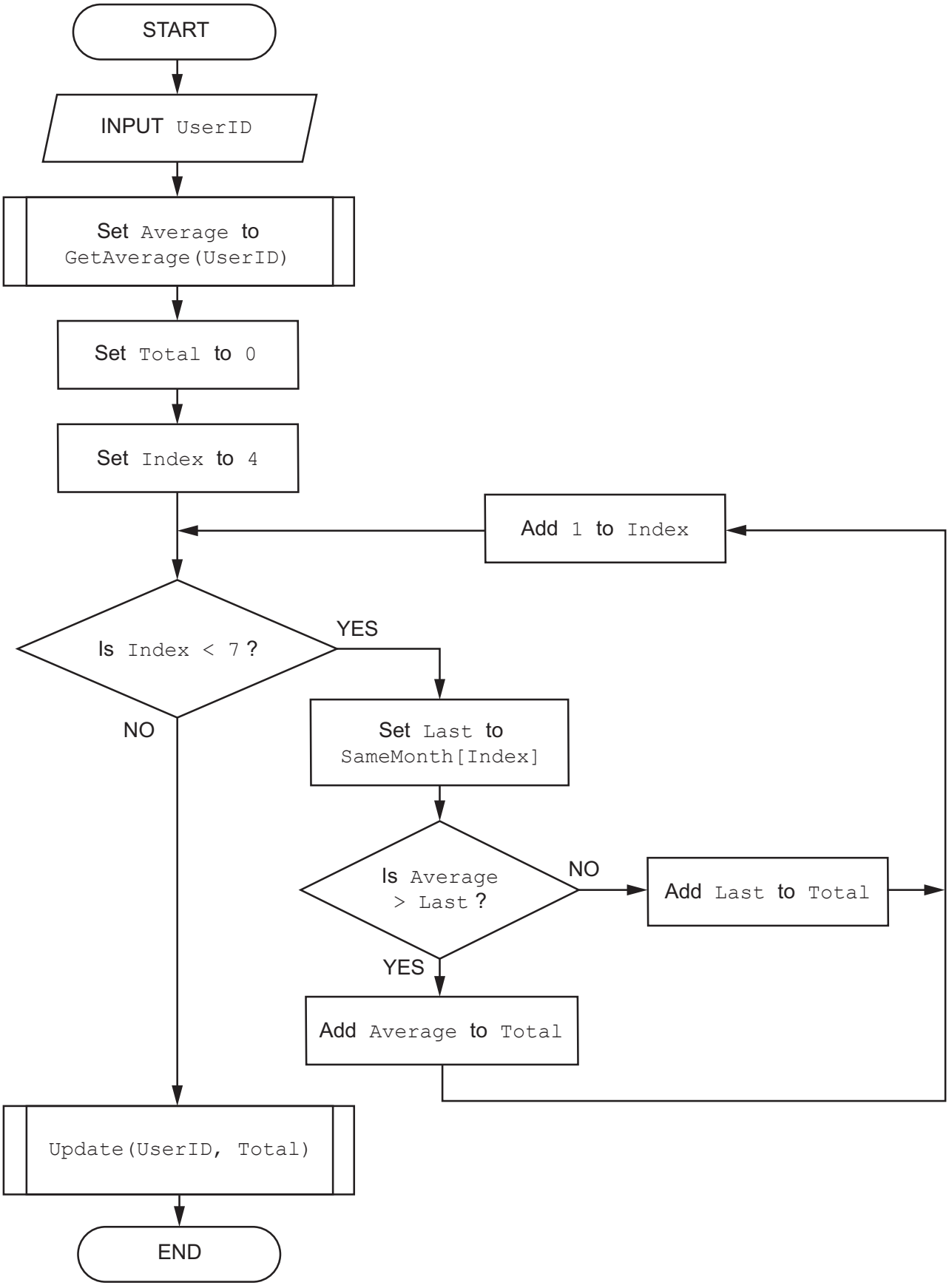
Variable	Type	Example value
FlagDay	DATE	23/04/2004
CharList	STRING	"ABCDEF"
Count	INTEGER	29

Complete the table by evaluating each expression using the example values.

Expression	Evaluation
MID(CharList, MONTH(FlagDay), 1)	
INT(Count / LENGTH(CharList))	
(Count >= 29) AND (DAY(FlagDay) > 23)	

[3]

4 The program flowchart represents a simple algorithm.



(a) Write the equivalent pseudocode for the algorithm represented by the flowchart.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) Give the name of the iterative construct in the flowchart.

..... [1]

5 Examine the following pseudocode.

```
IF A = TRUE THEN
   IF B = TRUE THEN
      IF C = TRUE THEN
         CALL Sub1()
      ELSE
         CALL Sub2()
      ENDIF
   ENDIF
ELSE
   IF B = TRUE THEN
      IF C = TRUE THEN
        CALL Sub4()
      ELSE
        CALL Sub3()
      ENDIF
   ELSE
      IF C = FALSE THEN
        CALL Sub3()
      ELSE
        CALL Sub4()
      ENDIF
   ENDIF
ENDIF
```

A programmer wants to re-write the pseudocode as **four** separate `IF...THEN...ENDIF` statements, each containing a single `CALL` statement. This involves writing a single, simplified logic expression as the condition in each statement.

Write the amended pseudocode.

1 .....

.....

.....

2 .....

.....

.....

3 .....

.....

.....

4 .....

.....

.....

[4]

6 **(a)** The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is n × (n−1) × ... × 2 × 1

For example, the factorial of 5 is 5 × 4 × 3 × 2 × 1 = 120

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter
- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return −1 if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	−1
6	3
1	1

`FindBaseNumber(12)` will return −1 because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

[7]

7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	"Undeclared variable"
4	60	"Type mismatch in assignment"
…		
500	999	<Undefined>

Note:

- There may be less than 500 error numbers so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value `999`. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present, for example, there may be no error code `31`.

The teacher has defined two program modules as follows:

Module	Description

`OutputError()`	• takes two parameters as integers:
	  ○ a line number in the student's program
	  ○ an error number
	• searches for the error number in the `ErrCode` array:
	  ○ if found, outputs the corresponding error description and the line number, for example:
	  `"Bracket mismatch on line 34"`
	  ○ if not found, outputs the line number and a warning, for example:
	  `"Unknown error on line 34"`

`SortArrays()`	sorts the arrays into ascending order of `ErrCode`

(a) Write **efficient** pseudocode for module `OutputError()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) Write an **efficient** bubble sort algorithm in pseudocode for module `SortArrays()`.

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** An algorithm includes a number of complex calculations. A programmer is writing a program to implement the algorithm and decides to use library routines to provide part of the solution.

State **three** possible benefits of using library routines in the development of the program.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

(c) The pseudocode statements in the following table may contain errors.

State the error in each case or write 'NO ERROR' if the statement contains no error.

Assume that any variables used are of the correct type for the given function.

Statement	Error
`IF EMPTY ← "" THEN`	
`Status ← IS_NUM(-23.4)`	
`X ← STR_TO_NUM("37") + 5`	
`Y ← STR_TO_NUM("37" + "5")`	

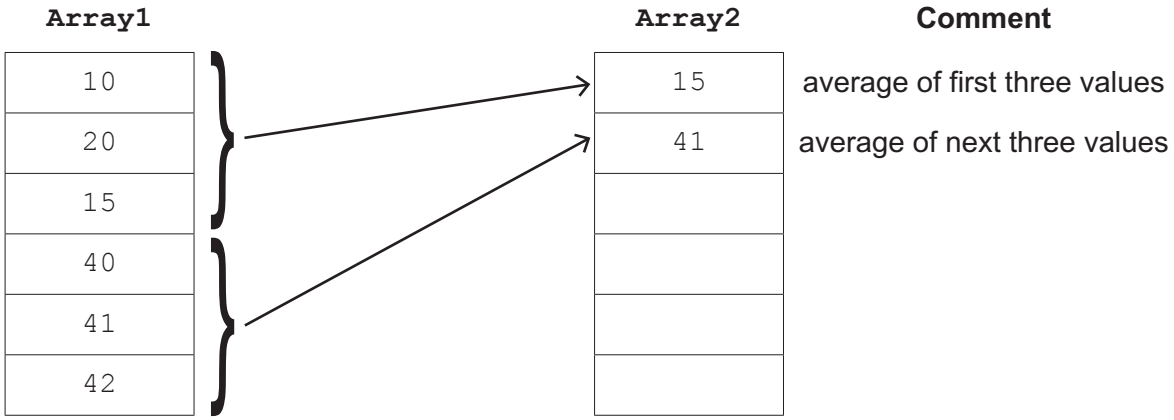
[4]

5 A program uses two 1D arrays of type integer. `Array1` contains 600 elements and `Array2` contains 200 elements.

`Array1` contains sample values read from a sensor. The sensor always takes three consecutive samples and all of these values are stored in `Array1`.

A procedure `Summarise()` will calculate the average of three consecutive values from `Array1` and write the result to `Array2`. This will be repeated for all values in `Array1`.

The diagram below illustrates the process for the first six entries in `Array1`.



Write pseudocode for the procedure `Summarise()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

6 The following pseudocode algorithm attempts to check whether a string is a valid email address.

```
FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
   DECLARE Index, Dots, Ats, Others : INTEGER
   DECLARE NextChar : CHAR
   DECLARE Valid : BOOLEAN

   Index ← 1
   Dots ← 0
   Ats ← 0
   Others ← 0
   Valid ← TRUE

   REPEAT
      NextChar ← MID(InString, Index, 1)
      CASE OF NextChar
         '.' : Dots ← Dots + 1
         '@' : Ats ← Ats + 1
               IF Ats > 1 THEN
                  Valid ← FALSE
               ENDIF
         OTHERWISE : Others ← Others + 1
      ENDCASE

      IF Dots > 1 AND Ats = 0 THEN
         Valid ← FALSE
      ELSE
         Index ← Index + 1
      ENDIF

   UNTIL Index > LENGTH(InString) OR Valid = FALSE

   IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
      Valid ← FALSE
   ENDIF

   RETURN Valid

ENDFUNCTION
```

(a) Part of the validation is implemented by the line:

```
IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
```

State the values that would result in the condition evaluating to `TRUE`.

.....

.....

..... [1]

(b) (i) Complete the trace table by dry running the function when it is called as follows:

```
Result ← IsValid("Liz.123@big@net")
```

Index	NextChar	Dots	Ats	Others	Valid

[5]

(ii) State the value returned when `IsValid()` is called using the expression shown in **part (b)(i)**.

..... [1]

7 A simple arithmetic expression is stored as a string in the format:

<Value1><Operator><Value2>

An operator character is one of the following: `'+'` `'-'` `'*'` `'/'`

Example arithmetic expression strings:

```
"803+1904"
"34/7"
```

(a) A procedure `Calculate()` will:

- take an arithmetic expression string as a parameter
- evaluate the expression
- output the result.

Assume:

- the string contains only numeric digits and a single operator character
- Value1 and Value2 represent integer values
- Value1 and Value2 are unsigned (they will not be preceded by `'+'` or `'-'`).

(i) Write pseudocode for the procedure `Calculate()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(ii) `Calculate()` is changed to a function that returns the value of the evaluated expression.

Write the header for the function in pseudocode.

.....

..... [1]

(b) A string representing an arithmetic expression could be in the correct format but be impossible to evaluate.

Give an example of a correctly formatted string and explain why evaluation would be impossible.

Example string .....

Explanation .....

.....

.....
[2]

8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
`CheckFile()`	• takes the name of an existing project file as a parameter of type string • returns `TRUE` if the file is valid (it contains at least 10 non-blank lines), otherwise returns `FALSE`

(a) Write pseudocode for module `CheckFile()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Further modules are defined as follows:

Module	Description
`CheckLine()`	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
`CountErrors()`	• takes two parameters: ○ the name of a project file as a string ○ the maximum number of errors as an integer • uses `CheckFile()` to test the project file. Outputs an error message and ends if the project file is not valid • calls `CheckLine()` for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

(b) `CountErrors()` is called to check the project file `Jim01Prog.txt` and to stop if more than 20 errors are found.

Write the pseudocode statement for this call.

.....

..... [2]

(c) Write pseudocode for module `CountErrors()`. Assume `CheckFile()` and `CheckLine()` have been written and can be used in your solution.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

2 **(a)** An algorithm to sort a 1D array into ascending order is described as follows:

- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

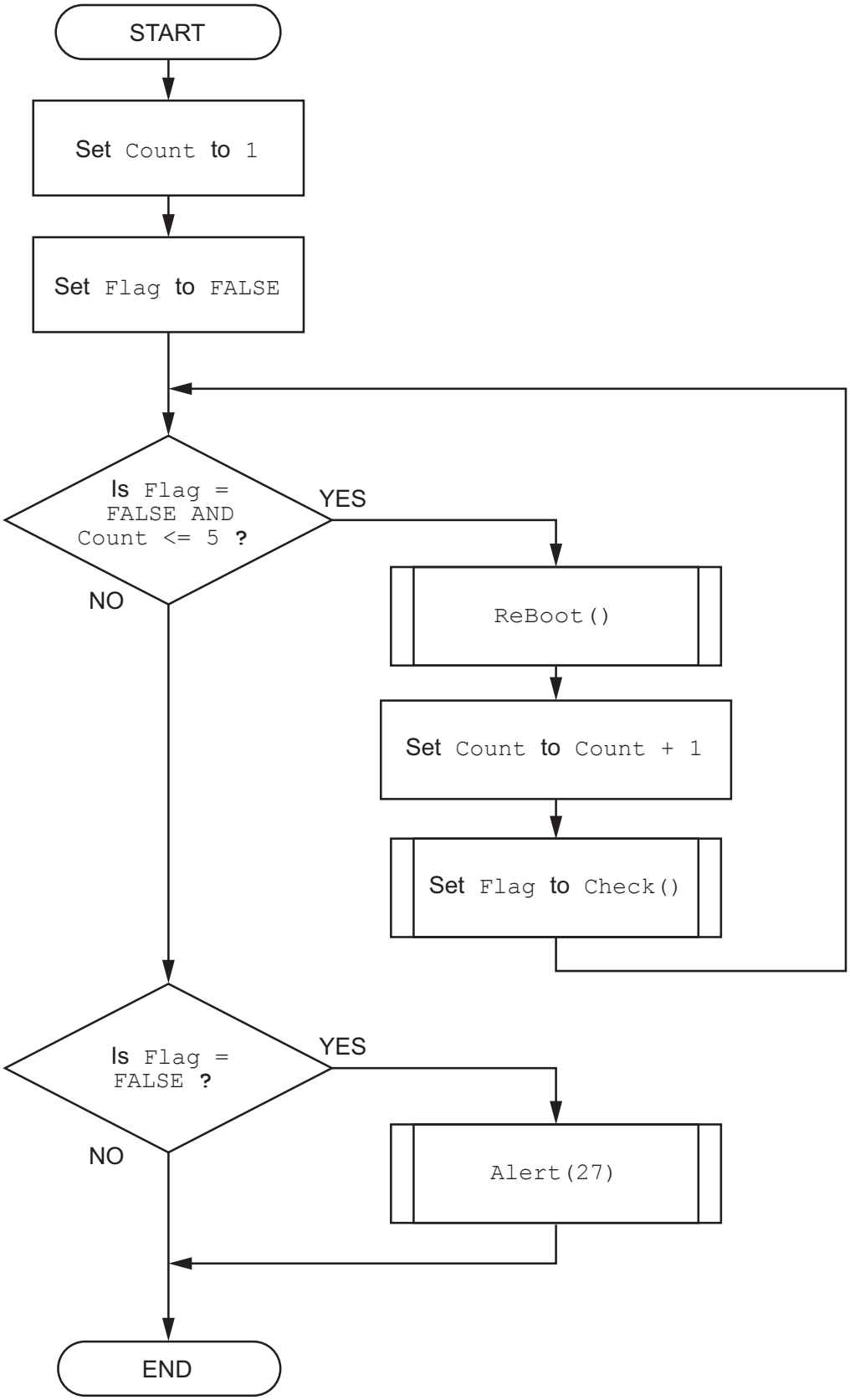
.....

.....

.....

..... [6]

(b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	`"CJL404"`
Host ID	`"Lib01"`
Time and date	`"08:30, June 01, 2021"`

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

(a) Suggest a suitable separator character **and** give a reason for your choice.

Character .....

Reason .....

.....

[2]

(c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
  - add the value of the array element to the existing text file `LogFile`
  - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

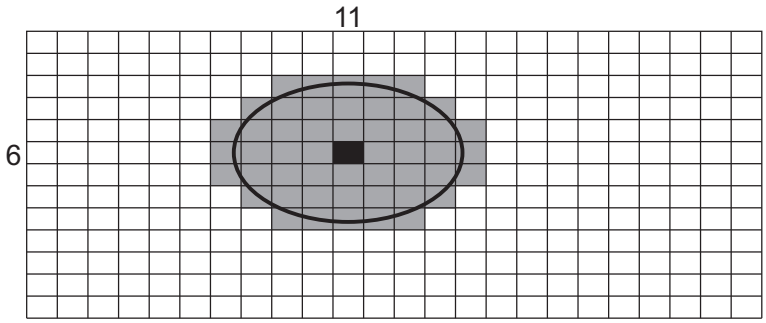
..... [7]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
`SetRow()` (generates test data)	• Called with three parameters of type `INTEGER`: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, `SetRow(3, 8, 5)` will give row 3 as in the diagram shown.
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1
`SearchInCol()`	• Takes two parameters of type `INTEGER`: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns −1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

.....

.....

.....

.....

.....

.....

.....

..... [8]

(c) The following new module is introduced:

Module	Description
`GetCentreCol()`	• Called with a row number as an `INTEGER` • Uses `SearchInRow()` to find the first and last columns in the given row which have an array element set to 1 • Returns the index of the column midway between the first and last columns • Returns −1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....[6]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** A programmer applies decomposition to a problem that she has been asked to solve.

Describe decomposition.

.....

.....

.....

.....

.....

..... [2]

(c) Complete the pseudocode expressions so that they evaluate to the values shown.

Any functions and operators used must be defined in the **insert**.

Expression	Evaluates to
..... ('C')	67
2 * ..... ("27")	54
..... (27 / .....)	13
"Sub" & ..... ("Abstraction" , ... , ...)	"Subtract"

[4]

(d) Evaluate the expressions given in the following table. The variables have been assigned values as follows:

```
PumpOn ← TRUE
PressureOK ← TRUE
HiFlow ← FALSE
```

Expression	Evaluates to
PressureOK AND HiFlow	
PumpOn OR PressureOK	
NOT PumpOn OR (PressureOK AND NOT HiFlow)	
NOT (PumpOn OR PressureOK) AND NOT HiFlow	

[2]

3 A programmer is writing a program to help manage clubs in a school.

Data will be stored about each student in the school and each student may join up to three clubs.

The data will be held in a record structure of type `Student`.

The programmer has started to define the fields that will be needed as shown in the following table.

Field	Typical value	Comment
`StudentID`	`"CF1234"`	Unique to each student
`Email`	`"Carmen47@xyzmail.com"`	Contains letters, numbers and certain symbols
`Club_1`	`1`	Any value in the range 1 to 99 inclusive
`Club_2`	`14`	Any value in the range 1 to 99 inclusive
`Club_3`	`27`	Any value in the range 1 to 99 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Student`.

.....

.....

.....

.....

.....

.....

..... [3]

(ii) A 1D array `Membership` containing 3000 elements will be used to store the student data.

Write pseudocode to declare the `Membership` array.

.....

..... [2]

(b) A procedure `GetIDs()` will:

- prompt and input the number of a club
- output the `StudentID` of all the students who are members of that club
- output a count of all students in the given club.

Write pseudocode for the procedure `GetIDs()`.

.....

.....

.....

4 The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```
10  PROCEDURE Check(InString : STRING)
11     DECLARE Odds, Evens, Index : INTEGER
12
13     Odds ← 0
14     Evens ← 0
15     Index ← 1
16
17     WHILE Index <= LENGTH(InString)
18        IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19           Odds ← Odds + 1
20        ELSE
21           Evens ← Evens + 1
22        ENDIF
23        Index ← Index + 1
24     ENDWHILE
25
26     CALL Result(Odds, Evens)
27  ENDPROCEDURE
```

(a) Complete the following table by giving the answers, using the given pseudocode.

	Answer
A line number containing a variable being incremented	
The type of loop structure	
The number of functions used	
The number of parameters passed to `STR_TO_NUM()`	
The name of a procedure other than `Check()`	

[5]

(c) **(i)** The loop structure used in the pseudocode is not the most appropriate.

State a more appropriate loop structure **and** justify your choice.

Loop structure .....

Justification .....

.....

.....

[2]

(ii) The appropriate loop structure is now used. Two lines of pseudocode are changed and two lines are removed.

Write the line numbers of the two lines that are removed.

.....

..... [1]

5 A company has several departments. Each department stores the name, email address and the status of each employee in that department in its own text file. All text files have the same format.

Employee details are stored as three separate data strings on three consecutive lines of the file. An example of the first six lines of one of the files is as follows:

File line	Comment
1	First employee name
2	First email address
3	First employee status
4	Second employee name
5	Second email address
6	Second employee status

A procedure `MakeNewFile()` will:

- take three parameters as strings:
  - an existing file name
  - a new file name
  - a search status value
- create a new text file using the new file name
- write all employee details to the new file where the employee status is **not** equal to the search status value
- count the number of sets of employee details that were in the original file
- count the number of sets of employee details that were written to the new file
- produce a summary output.

An example summary output is as follows:

```
File Marketing contained 54 employee details
52 employee sets of details were written to file NewMarketingList
```

(a) Write pseudocode for the procedure `MakeNewFile()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

(ii) State **one** advantage **and one** disadvantage of the alternative format.

Advantage .....

.....

Disadvantage .....

.....

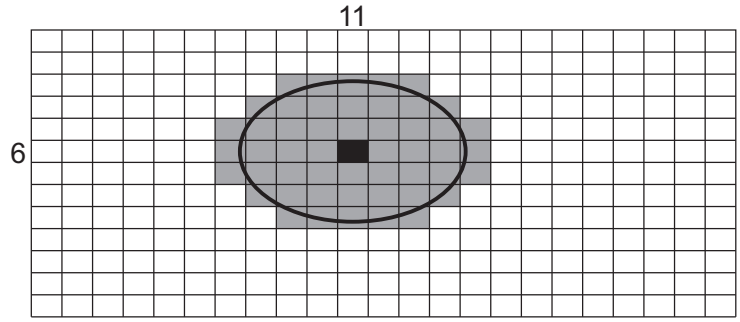
[2]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
`FirstRowSet()`	• Searches for the first row that has an array element set to 1 • Returns the index of that row (1 is the first row) • Returns −1 if there are no elements set to 1
`LastRowSet()`	• Searches for the last row that has an array element set to 1 • Returns the index of that row • Returns −1 if there are no elements set to 1
`FirstColSet()`	• Searches for the first column that has an array element set to 1 • Returns the index of that column (1 is the first column) • Returns −1 if there are no elements set to 1
`LastColSet()`	• Searches for the last column that has an array element set to 1 • Returns the index of that column • Returns −1 if there are no elements set to 1

(a) Write efficient pseudocode for the module `FirstRowSet()`.

[7]

(b) Describe a feature of your solution to **part (a)** that indicates the pseudocode represents an efficient algorithm.

.....

.....

.....

..... [2]

(c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

(i) Outline the changes needed to convert one of the existing modules into this single module.

.....

.....

.....

.....

.....

..... [2]

(ii) Give one possible advantage **and** one possible disadvantage of combining the four searches into a single module.

Advantage .....

.....

Disadvantage .....

.....
[2]

(d) An additional module `GetCentre()` is defined as follows:

Module	Description
`GetCentre()`	• Calculates the coordinates of the centre point • Uses the four modules: `FirstRowSet()`, `LastRowSet()`, `FirstColSet()`, `LastColSet()` • Assigns values to `CentreRow` and `CentreCol` • Sets `CentreRow` to −1 if no screen touch is detected

Write pseudocode for the module `GetCentre()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

2 **(a)** An algorithm to sort a 1D array into ascending order is described as follows:

- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

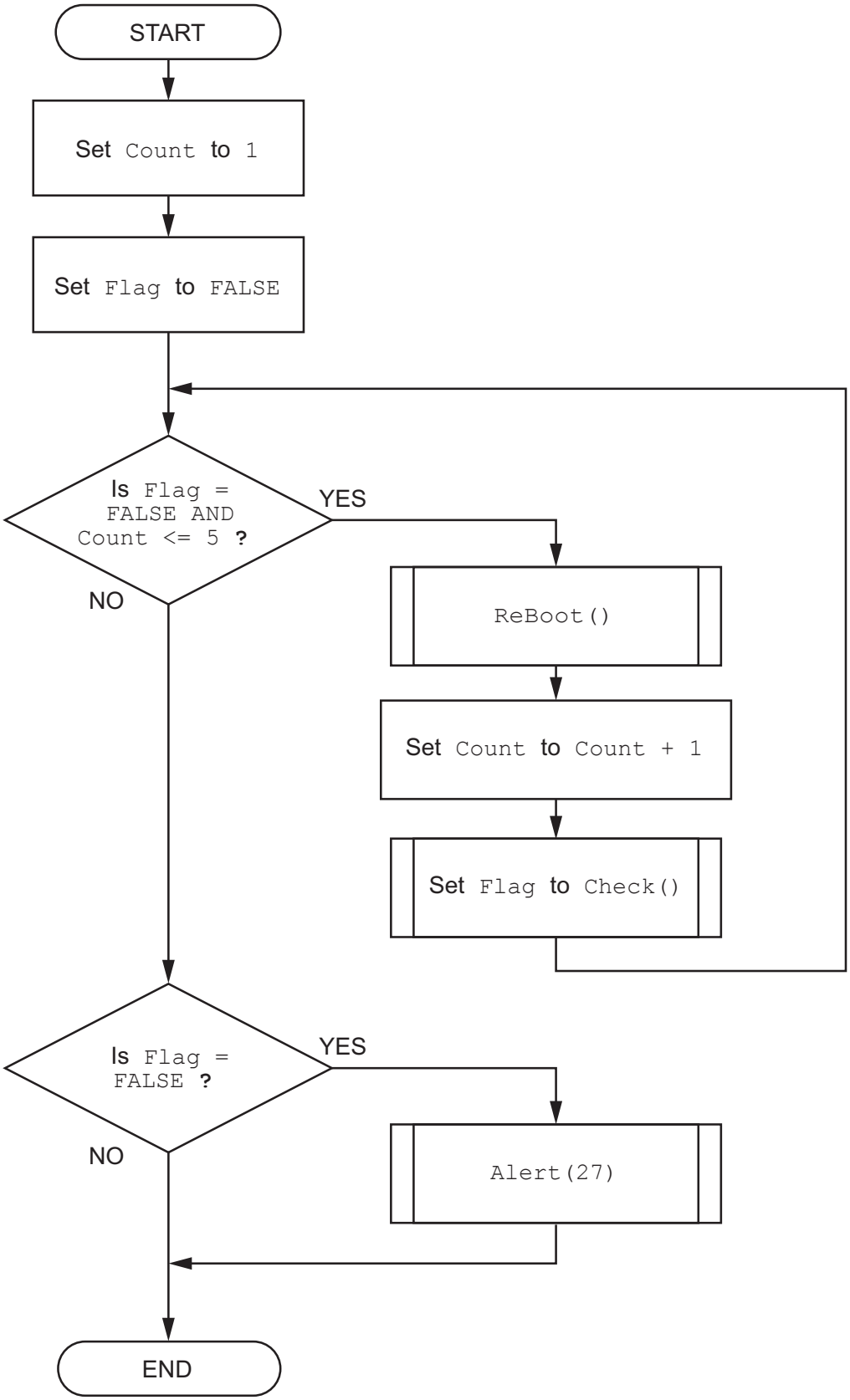
.....

.....

.....

..... [6]

(b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	`"CJL404"`
Host ID	`"Lib01"`
Time and date	`"08:30, June 01, 2021"`

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

(a) Suggest a suitable separator character **and** give a reason for your choice.

Character .....

Reason .....

.....

[2]

(c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
  - add the value of the array element to the existing text file `LogFile`
  - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

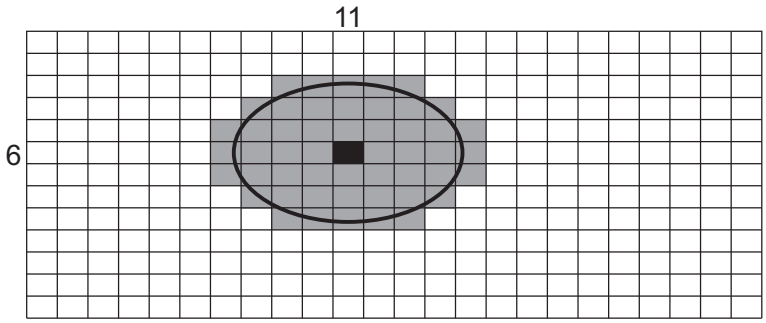
..... [7]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
`SetRow()` (generates test data)	• Called with three parameters of type `INTEGER`: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, `SetRow(3, 8, 5)` will give row 3 as in the diagram shown.
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1
`SearchInCol()`	• Takes two parameters of type `INTEGER`: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns −1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

.....

.....

.....

.....

.....

.....

.....

..... [8]

(c) The following new module is introduced:

Module	Description
`GetCentreCol()`	• Called with a row number as an `INTEGER` • Uses `SearchInRow()` to find the first and last columns in the given row which have an array element set to 1 • Returns the index of the column midway between the first and last columns • Returns −1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....[6]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club		
`FALSE`	A value to indicate whether an existing member of the club lives at the same address		
`19/02/1983`	When the member joined the football club		
`1345`	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

(b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
`Result ← 2 & 4`	
`SubString ← MID("pseudocode", 4, 1)`	
`IF x = 3 OR 4 THEN`	
`Result ← Status AND INT(x/2)`	
`Message ← "Done" + LENGTH(MyString)`	

[5]

2 **(a)** Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
`UpdateLoan()`	• Calls either `LoanExtend()` or `LoanReturn()`
`LoanExtend()`	• Called with parameters `LoanID` and `BookID` • Calls `CheckReserve()` to see whether the book has been reserved for another library user • Returns `TRUE` if the loan has been extended, otherwise returns `FALSE`
`CheckReserve()`	• Called with `BookID` • Returns `TRUE` if the book has been reserved, otherwise returns `FALSE`
`LoanReturn()`	• Called with parameters `LoanID` and `BookID` • Returns a `REAL` (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
`LoanReturn()`	Called with parameters `LoanID`, `BookID` and `Fine` The module code checks whether the book has been returned on time and then assigns a new value to `Fine`

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

.....

..... [2]

3 **(a)** A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

(i) Explain how the program could use text files to speed up the process.

.....

.....

.....

.....

.....

..... [2]

(b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

4 Study the following pseudocode. Line numbers are for reference only.

```
10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""      // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24      ThisChar ← MID(Name, Index, 1)
25      IF Flag = TRUE THEN
26         NewName ← NewName & UCASE(ThisChar)
27         IF ThisChar <> SPACECHAR THEN
28            Flag ← FALSE
29         ENDIF
30      ELSE
31         NewName ← NewName & ThisChar
32      ENDIF
33      IF ThisChar = SPACECHAR THEN
34         Flag ← TRUE
35      ENDIF
36      Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION
```

(a) Complete the trace table below by dry running the function when it is called as follows:

```
Result ← Convert("∇in∇a∇∇Cup")
```

Note: The symbol `'∇'` has been used to represent a space character.
Use this symbol for any space characters in the trace table.

The first row has been completed for you.

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				

[5]

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure .....

.....

Justification .....

.....

.....

[2]

(c) Two changes need to be made to the algorithm.

Change 1: Convert to lower case any character that is not the first character after a space.
Change 2: Replace multiple spaces with a single space.

(i) Change 1 may be implemented by modifying one line of the pseudocode.

Write the modified line.

.....

.....[1]

5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as a parameter, representing the number of a word in `FNString`. • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: if `FNString` contains the string `"hot and cold"`, `GetStart(3)` returns `9`
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

(a) Write pseudocode for the module `GetStart()`.

.....

.....

.....

.....

.....

.....

..... [7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

Write pseudocode for the module `GetWord()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
`Name`	`"Catherine"`	
`Index`	`100`	
`Modified`	`FALSE`	
`Holiday`	`25/12/2020`	

[4]

(ii) Evaluate each expression in the following table by using the initial data values shown in **part (a)(i)**.

Expression	Evaluates to
`Modified OR Index > 100`	
`LENGTH("Student: " & Name)`	
`INT(Index + 2.9)`	
`MID(Name, 1, 3)`	

[4]

(b) Each pseudocode statement in the following table contains an example of selection, assignment or iteration.

Put **one** tick (‘✓’) in the appropriate column for each statement.

Statement	Selection	Assignment	Iteration
`Index ← Index + 1`			
`IF Modified = TRUE THEN`			
`ENDWHILE`			

[3]

5 **(a)** A student is learning about arrays.

She wants to write a program to:

- declare a 1D array `RNum` of 100 elements of type `INTEGER`
- assign each element a random value in the range 1 to 200 inclusive
- count and output how many numbers generated were between 66 and 173 inclusive.

(i) Write pseudocode to represent the algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The following is a pseudocode function.

Line numbers are given for reference only.

```
01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03     DECLARE NextChar : CHAR
04     DECLARE OutString : STRING
05     DECLARE Counter : INTEGER
06
07     OutString ← ""
08
09     FOR Counter ← 1 TO LENGTH(InString)
10        NextChar ← MID(InString, Counter, 1)
11        NextChar ← LCASE(NextChar)
12        IF NOT((NextChar < 'a') OR (NextChar > 'z')) THEN
13           OutString ← OutString & NextChar
14        ENDIF
15     NEXT Counter
16
17     RETURN OutString
18
19  ENDFUNCTION
```

(i) Examine the pseudocode and complete the following table.

	Answer
Give a line number containing an example of an initialisation statement.	
Give a line number containing the start of a repeating block of code.	
Give a line number containing a logic operation.	
Give the number of parameters to the function `MID()`.	

[4]

(ii) Write a simplified version of the statement in line `12`.

.....

..... [2]

6 A procedure `CountVowels()` will:

- be called with a string containing alphanumeric characters as its parameter
- count and output the number of occurrences of each vowel (a, e, i, o, u) in the string
- count and output the number of occurrences of the other alphabetic characters (as a single total).

The string may contain both upper and lower case characters.

Each count value will be stored in a unique element of a global 1D array `CharCount` of type `INTEGER`. The array will contain six elements.

Write pseudocode for the procedure `CountVowels()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as its parameter, representing the number of a word in `FNString` • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: `GetStart(3)` applied to `"hot and cold"` returns `9`
`GetWord()`	• Called with the position of the first character of a word in `FNString` as its parameter • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`
`IgnoreWord()`	• Called with a `STRING` parameter representing a word • Searches for the word in the `IgnoreList` array • Returns `TRUE` if the word is found, otherwise returns `FALSE`
`GetInitials()`	• Processes the sequence of words in the full name one word at a time • Calls `GetStart()`, `GetWord()` and `IgnoreWord()` to process each word to form the new string • Outputs the new string

(a) Write pseudocode for the module `IgnoreWord()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) Write pseudocode for the module `GetInitials()`.

[8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club		
`FALSE`	A value to indicate whether an existing member of the club lives at the same address		
`19/02/1983`	When the member joined the football club		
`1345`	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

(b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
`Result ← 2 & 4`	
`SubString ← MID("pseudocode", 4, 1)`	
`IF x = 3 OR 4 THEN`	
`Result ← Status AND INT(x/2)`	
`Message ← "Done" + LENGTH(MyString)`	

[5]

2 **(a)** Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
`UpdateLoan()`	• Calls either `LoanExtend()` or `LoanReturn()`
`LoanExtend()`	• Called with parameters `LoanID` and `BookID` • Calls `CheckReserve()` to see whether the book has been reserved for another library user • Returns `TRUE` if the loan has been extended, otherwise returns `FALSE`
`CheckReserve()`	• Called with `BookID` • Returns `TRUE` if the book has been reserved, otherwise returns `FALSE`
`LoanReturn()`	• Called with parameters `LoanID` and `BookID` • Returns a `REAL` (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
`LoanReturn()`	Called with parameters `LoanID`, `BookID` and `Fine` The module code checks whether the book has been returned on time and then assigns a new value to `Fine`

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

.....

..... [2]

3 **(a)** A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

(i) Explain how the program could use text files to speed up the process.

.....

.....

.....

.....

.....

..... [2]

(b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

4 Study the following pseudocode. Line numbers are for reference only.

```
10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""      // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24      ThisChar ← MID(Name, Index, 1)
25      IF Flag = TRUE THEN
26         NewName ← NewName & UCASE(ThisChar)
27         IF ThisChar <> SPACECHAR THEN
28            Flag ← FALSE
29         ENDIF
30      ELSE
31         NewName ← NewName & ThisChar
32      ENDIF
33      IF ThisChar = SPACECHAR THEN
34         Flag ← TRUE
35      ENDIF
36      Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION
```

(a) Complete the trace table below by dry running the function when it is called as follows:

```
Result ← Convert("∇in∇a∇∇Cup")
```

Note: The symbol `'∇'` has been used to represent a space character.
Use this symbol for any space characters in the trace table.

The first row has been completed for you.

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				

[5]

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure .....

.....

Justification .....

.....

.....

[2]

(c) Two changes need to be made to the algorithm.

Change 1: Convert to lower case any character that is not the first character after a space.
Change 2: Replace multiple spaces with a single space.

(i) Change 1 may be implemented by modifying one line of the pseudocode.

Write the modified line.

.....

.....[1]

5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as a parameter, representing the number of a word in `FNString`. • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: if `FNString` contains the string `"hot and cold"`, `GetStart(3)` returns `9`
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

(a) Write pseudocode for the module `GetStart()`.

.....

.....

.....

.....

.....

.....

..... [7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

Write pseudocode for the module `GetWord()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]